

Rapport de Projet de Fin d'Études
5^e année – Ingénierie Informatique et Réseaux
Année universitaire 2025–2026

Sous le thème :

Migration Factory

**Conception et développement d'une plateforme
intelligente, extensible et gouvernée
pour l'automatisation des migrations logicielles**

Architecture multi-agents appliquée à Java/Spring Boot et Angular

Réalisé par :

HAMDAOUI ALI

Organisme d'accueil :

**CGI Technologies et
Solutions Maroc**

Encadré par :

Encadrant professionnel :

M. MOUTAI Ghassan

M. LAHOUIR Omar

Encadrant pédagogique :

M. NAFIL Khalid

Période académique : 2025–2026

Dédicace

À mes parents, pour leur confiance, leurs sacrifices et le soutien qu'ils m'ont toujours accordé. Ce parcours porte une part de leurs efforts et de leurs encouragements.

À ma famille et à mes proches, pour leur présence, leur patience et les mots qui m'ont aidé à avancer dans les périodes les plus exigeantes.

À celles et ceux qui, par leurs conseils, leur exemple ou leur bienveillance, ont nourri mon envie d'apprendre, de progresser et de mener mes engagements jusqu'au bout.

HAMDAOUI Ali

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma sincère gratitude à toutes les personnes qui ont contribué, directement ou indirectement, à sa réalisation.

Je remercie tout particulièrement **CGI Technologies et Solutions Maroc** pour son accueil et pour l'environnement professionnel mis à ma disposition. Cette expérience m'a permis de travailler sur des problématiques concrètes de modernisation applicative, d'automatisation et d'intelligence artificielle appliquée à l'ingénierie logicielle.

J'adresse une reconnaissance particulière à **M. NAFIL Khalid**, pour sa disponibilité, la précision de ses remarques et l'exigence constructive de son accompagnement. Ses retours m'ont permis de mieux structurer la démarche, de renforcer l'argumentation du rapport et d'améliorer la qualité de sa présentation.

Je remercie également mes encadrants et l'ensemble de l'équipe impliquée dans la *Migration Factory*. Les échanges autour de l'orchestration, des pipelines de migration, des agents spécialisés, de la validation et de l'expérience utilisateur ont contribué à faire évoluer le projet au-delà d'un prototype technique vers une plateforme plus générale et réutilisable.

Mes remerciements vont enfin à l'**École Marocaine des Sciences de l'Ingénieur**, à ses enseignants et à toutes les personnes qui ont participé à ma formation. Les connaissances acquises au cours de mon cursus ont constitué une base déterminante pour mener ce projet à la croisée de l'architecture logicielle, de l'automatisation, du cloud et de l'intelligence artificielle générative.

Résumé

La modernisation des applications reste souvent coûteuse en temps et en expertise, car elle combine analyse du projet, résolution de compatibilités, transformation du code, exécution des outils, tests et cycles répétés de diagnostic. Ce projet de fin d'études, réalisé au sein de **CGI Technologies et Solutions Maroc**, vise à répondre à cette difficulté par la conception d'une **Migration Factory intelligente, extensible et gouvernée**.

La solution repose sur une architecture multi-agents hybride et centralement orchestrée. Les agents interviennent sur les tâches nécessitant raisonnement ou adaptation, notamment l'analyse, la planification, la révision, la transformation agentique, la réparation et l'assistance. Les opérations sensibles et vérifiables restent confiées à des services déterministes qui contrôlent l'état du workflow, les transitions, les commandes, les écritures, les validations et la conservation des preuves. Le développeur conserve ainsi l'autorité sur les décisions importantes.

Deux déclinaisons indépendantes ont été développées. La première cible **Java/Spring Boot** et constitue l'implémentation la plus mature, avec une trajectoire progressive, des espaces de travail isolés, des validations Maven, une boucle de réparation gouvernée, un cockpit, un assistant et un reporting. Elle intègre également Azure OpenAI au moyen d'une configuration externalisée permettant d'associer des déploiements de modèles distincts aux rôles agentiques. La seconde déclinaison cible **Angular**. La preuve d'exécution retenue valide les transitions Angular 18 vers 19 et 19 vers 20; le stage 20 vers 21 est préparé mais non démarré, tandis que l'extension vers une trajectoire 11 vers 21 reste à compléter.

Les résultats montrent surtout l'intérêt de séparer raisonnement, décision et exécution afin d'améliorer la traçabilité, la reproductibilité et le contrôle du processus. En revanche, les gains globaux de temps, de coût et de productivité ne sont pas encore démontrés quantitativement sur un corpus représentatif. La contribution principale réside donc dans une architecture gouvernée qui automatise davantage la migration tout en maintenant des mécanismes déterministes et une supervision humaine.

Mots-clés : Migration logicielle, modernisation applicative, systèmes multi-agents, LLM, orchestration, Java, Spring Boot, Angular, Azure OpenAI, traçabilité, human-in-the-loop.

Abstract

Application modernization often remains costly in time and expertise because it combines project analysis, compatibility resolution, code transformation, tool execution, testing, and repeated diagnostic cycles. This final-year engineering project, carried out at **CGI Technologies and Solutions Morocco**, addresses this challenge through the design of an **intelligent, extensible, and governed Migration Factory**.

The solution relies on a hybrid, centrally orchestrated multi-agent architecture. Agents handle tasks that require reasoning or adaptation, including analysis, planning, review, agentic transformation, repair, and assistance. Sensitive and verifiable operations remain under deterministic services that control workflow state, transitions, commands, writes, validations, and evidence preservation. The developer therefore retains authority over important decisions.

Two independent implementations were developed. The first targets **Java/Spring Boot** and is the most mature implementation, with a progressive migration path, isolated workspaces, Maven-based validation, a governed repair loop, a cockpit, an assistant, and reporting capabilities. It also integrates Azure OpenAI through externalized configuration that maps distinct model deployments to agent roles. The second implementation targets **Angular**. The retained execution evidence validates the Angular 18-to-19 and 19-to-20 transitions; the 20-to-21 stage is prepared but has not started, while the broader 11-to-21 trajectory remains to be completed.

The results mainly demonstrate the value of separating reasoning, decision-making, and execution in order to improve traceability, reproducibility, and process control. However, overall gains in time, cost, and productivity have not yet been quantitatively demonstrated on a representative corpus. The main contribution is therefore a governed architecture that increases migration automation while preserving deterministic safeguards and human supervision.

Keywords : Software migration, application modernization, multi-agent systems, LLM, orchestration, Java, Spring Boot, Angular, Azure OpenAI, traceability, human-in-the-loop.

ملخص

تظل عملية تحديث التطبيقات البرمجية مكلفة من حيث الوقت والخبرة، لأنها تجمع بين تحليل المشروع، وحل مشكلات توافق الإصدارات، وتعديل الشيفرة، وتشغيل أدوات البناء والاختبار، وتكرار دورات التشخيص والتصحيح. يهدف مشروع نهاية الدراسة هذا، المنجز داخل شركة CGI Technologies et Solutions Maroc ، إلى معالجة هذه الصعوبة من خلال تصميم منصة Migration Factory ذكية وقابلة للتوسع ومحكومة بآليات واضحة للرقابة.

تعتمد الحلول المقترحة على معمارية هجينة متعددة الوكلاء ومنظمة مركزيا. تتدخل الوكلاء في المهام التي تحتاج إلى التحليل أو التكيف، مثل تحليل المشروع، والتخطيط، والمراجعة، والتحويل المعتمد على النماذج، والإصلاح، والمساعدة التفاعلية. أما العمليات الحساسة والقابلة للتحقق فتبقى تحت مسؤولية خدمات حتمية تتحكم في حالة سير العمل، والانتقالات، والأوامر، وعمليات الكتابة، والتحقق، وحفظ الأدلة التقنية. وبذلك يحتفظ المطور بسلطة القرار في المراحل الحساسة.

تم تطوير تطبيقين مستقلين. يستهدف الأول Java/Spring Boot ، وهو الأكثر نضجا حاليا، إذ يدعم مسار ترحيل تدريجيا، وبيئات عمل معزولة، والتحقق بواسطة Maven ، وحلقة إصلاح محكمة، وواجهة متابعة، ومساعدة، وتقارير نهائية. كما يدمج Azure OpenAI عبر إعدادات خارجية تسمح بربط عمليات النشر المختلفة بأدوار الوكلاء. أما التطبيق الثاني فيستهدف Angular . وتظهر أدلة التنفيذ المعتمدة نجاح الانتقال من Angular 18 إلى Angular 19 ثم من 19 إلى 20 ، بينما تم إعداد مرحلة 20 إلى 21 دون بدء تنفيذها بعد، وما زال المسار الأشمل من Angular 11 إلى 21 يحتاج إلى استكمال.

تبين النتائج أساسا أهمية الفصل بين الاستدلال، واتخاذ القرار، والتنفيذ من أجل تحسين قابلية التتبع، وإعادة الإنتاج، والتحكم في عملية الترحيل. ومع ذلك، لم تثبت بعد بصورة كمية المكاسب الإجمالية في الوقت والتكلفة والإنتاجية على مجموعة ممثلة من المشاريع. لذلك تتمثل المساهمة الرئيسية في معمارية محكمة توسع مستوى الأتمتة مع الحفاظ على الضمانات الحتمية والإشراف البشري.

الكلمات المفتاحية: ترحيل البرمجيات، تحديث التطبيقات، الأنظمة متعددة الوكلاء، النماذج اللغوية الكبيرة، التنسيق، Java ، Spring Boot ، Angular ، Azure OpenAI ، قابلية التتبع، الإنسان ضمن الحلقة.

Table des matières

Dédicace	i
Remerciements	ii
Résumé	iii
Abstract	iv
Résumé en arabe	v
Table des matières	ix
Liste des figures	x
Liste des tableaux	xi
Liste des acronymes	xii
Introduction générale	1
1 Contexte général du projet, analyse des besoins et spécification du système	3
Introduction	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 Le groupe CGI	3
1.1.2 CGI Technologies et Solutions Maroc	4
1.2 Présentation du projet	4
1.2.1 Domaine et finalité	4
1.2.2 Utilisateurs cibles	5
1.2.3 Classification des acteurs et composants	5
1.2.4 Fonctionnalités existantes	6
1.2.5 Genèse et évolution générale du projet	6
1.3 Problématique traitée	7
1.3.1 Constats et besoin de coordination	7
1.3.2 Limites d'une architecture monolithique ou d'un agent unique .	7
1.3.3 Problématique générale	7
1.4 Besoins fonctionnels	7

1.5	Besoins non fonctionnels	8
1.6	Objectifs du projet	9
1.6.1	Objectif général	9
1.6.2	Objectifs spécifiques	9
1.7	Méthodologie générale	9
1.7.1	Organisation Agile du projet	9
1.7.2	Planification prévisionnelle et rétrospective	10
1.7.3	Itérations, incréments et jalons de validation	12
1.7.4	Cycle technique	13
1.8	Contraintes d'intégration des modèles dans l'environnement CGI	14
	Conclusion	16
2	Cadrage conceptuel et état de l'art	17
	Introduction	17
2.1	Agents intelligents	17
2.1.1	Définition et interaction avec l'environnement	17
2.1.2	Propriétés	18
2.1.3	Types d'agents	18
2.1.4	Agent, LLM, service et outil	19
2.2	Systèmes multi-agents	19
2.2.1	Définition	19
2.2.2	Coordination, communication et délégation	20
2.2.3	Avantages et limites	20
2.3	Orchestration d'agents	20
2.3.1	Notions à distinguer	20
2.3.2	Modèles d'organisation	20
2.4	Plateformes et architectures intelligentes	21
2.5	Migration logicielle : motivations, types et processus	21
2.5.1	Définitions et motivations	21
2.5.2	Types et processus	22
2.5.3	Spécificités Java/Spring Boot et Angular	22
2.6	Étude de l'existant et travaux proches	22
2.6.1	Processus manuel de référence	22
2.6.2	Corpus étudié	23
2.6.3	Comparaison synthétique	23
2.6.4	Carte de positionnement	25
2.7	Positionnement de la Migration Factory et choix d'intégration	26
2.7.1	Synthèse du positionnement	26
2.7.2	Modèles managés et déploiements d'entreprise	26

Conclusion	27
3 Conception de l'architecture multi-agents	28
Introduction	28
3.1 Cadre de conception et choix du modèle SMA	28
3.1.1 Points de vue architecturaux	28
3.1.2 Choix du modèle SMA	29
3.2 Organisation des agents et de l'architecture	31
3.2.1 Agents intelligents retenus	31
3.2.2 Services déterministes	31
3.2.3 Matrice des rôles et de l'autonomie	32
3.2.4 Architecture globale	32
3.3 Communication, état et orchestration	33
3.3.1 Communication et contrats	33
3.3.2 Conception de l'orchestrateur	34
3.3.3 États, mémoire et contexte	34
3.4 Workflows, supervision et reprise	35
3.4.1 Workflows spécialisés	35
3.4.2 Supervision et réparation gouvernée	36
3.4.3 Gestion des erreurs et replanification	36
3.5 Passerelle d'accès aux modèles et routage par rôle	36
Conclusion	38
4 Architecture et implémentation du système	39
Introduction	39
4.1 Environnement technique	39
4.1.1 Technologies communes	39
4.1.2 Environnement Java/Spring Boot	40
4.1.3 Environnement Angular	40
4.2 Évolution technique de la solution Java/Spring Boot	41
4.2.1 Première version : preuve de concept en ligne de commande	41
4.2.2 Consolidation du prototype CLI	41
4.2.3 Passage à la Control Tower full stack	41
4.2.4 Consolidation de la V1 gouvernée	41
4.3 Structure simplifiée des dépôts	42
4.3.1 Dépôt Java/Spring Boot	43
4.3.2 Dépôt Angular	43
4.3.3 Déploiement local	43
4.4 Implémentation des agents	44

4.4.1	Agent d'analyse	44
4.4.2	Agent de révision de l'analyse	44
4.4.3	Agent de planification	45
4.4.4	Agent de révision du plan	45
4.4.5	Agent de transformation	46
4.4.6	Proposeur de réparation	46
4.4.7	Agent de révision de la réparation	47
4.4.8	Assistant de migration	47
4.5	Services déterministes et orchestrateur	48
4.6	Backend, persistance et interface de supervision	49
4.6.1	Backend, persistance et preuves	49
4.6.2	Interface de supervision et captures	49
4.7	Robustesse, sécurité et validation technique	52
4.7.1	Authentification, sécurité et limites d'exploitation	52
4.7.2	Difficultés techniques et corrections	53
4.7.3	Preuves techniques disponibles	53
4.8	Choix d'Azure OpenAI et configuration des modèles du client	54
4.8.1	Configuration observée dans la déclinaison Java	54
4.8.2	Chaîne d'appel et contrôle préalable	55
4.8.3	Protection des informations sensibles	55
	Conclusion	56
5	Évaluation, démonstrations et résultats	57
	Introduction	57
5.1	Objectifs et protocole d'évaluation	57
5.2	Critères et métriques	58
5.3	Validation progressive par les démonstrations CGI	58
5.4	Résultats Java/Spring Boot	60
5.5	Résultats Angular et extension	61
5.5.1	État actuel et résultats observés	61
5.5.2	Extension visée d'Angular 11 vers Angular 21	62
5.6	Analyse qualitative par caractéristique de qualité	62
5.6.1	Évaluation de l'intégration Azure OpenAI	65
5.7	Comparaison avec le processus manuel	67
5.8	Limites de l'évaluation	69
	Conclusion	69
	Conclusion générale	71
	Bibliographie	73

Table des figures

1.1	Logo de CGI	3
1.2	Vue de contexte et classification des acteurs et composants	5
1.3	Frise chronologique générale du projet	6
1.4	Dépendances et boucles de coordination d'une migration	7
1.5	Cycle itératif de conception, réalisation et validation	14
1.6	Principe d'intégration des déploiements Azure contrôlés par CGI	15
2.1	Interaction générale entre un agent et son environnement	17
2.2	Vue simplifiée d'un SMA spécialisé	19
2.3	Architecture en couches d'une plateforme agentique	21
2.4	Processus général d'une migration applicative	22
2.5	Boucle typique d'une migration conduite manuellement	23
2.6	Regroupement des approches, plateformes et standards étudiés	25
3.1	Modèle hybride retenu pour la Migration Factory	30
3.2	Architecture globale du système multi-agents	33
3.3	Séquence simplifiée entre agents, orchestrateur et services	34
3.4	Logique générale de l'orchestrateur	34
3.5	Machine à états simplifiée d'une migration	35
3.6	Boucle de réparation gouvernée	36
3.7	Passerelle Azure OpenAI et routage des déploiements par rôle	37
4.1	Évolution technique de la solution Java/Spring Boot	42
4.2	Déploiement local des prototypes	44
4.3	Séquencement principal implémenté par l'orchestrateur	48
4.4	Première version Java/Spring Boot utilisée en ligne de commande	50
4.5	Control Tower Java/Spring Boot	50
4.6	Assistant conversationnel de la Control Tower	51
4.7	Écran de réparation gouvernée de la V1 Java	51
4.8	Première démonstration d'avancement Angular	52
4.9	Chaîne d'appel Azure OpenAI dans le backend Java	55
5.1	Protocole d'évaluation retenu	57
5.2	Trajectoire progressive visée pour Angular 11 vers Angular 21	62

Liste des tableaux

1.1	Parties prenantes du projet	4
1.2	Besoins fonctionnels et maturité des deux systèmes	7
1.3	Exigences non fonctionnelles et critères de vérification	8
1.4	Diagramme de Gantt prévisionnel reconstitué du projet Migration Factory . .	11
1.5	Diagramme de Gantt rétrospectif du projet Migration Factory	12
1.6	Synthèse des principales itérations et des incréments	12
1.7	Exigences liées à l'intégration Azure OpenAI	15
2.1	Types d'agents et correspondance avec la Migration Factory	18
2.2	Orchestration, chorégraphie, coordination et planification	20
2.3	Comparaison des travaux et plateformes proches	24
2.4	Modes d'intégration des modèles dans une plateforme d'entreprise	27
3.1	Points de vue retenus pour la description de l'architecture	29
3.2	Autonomie et limites des agents	32
3.3	Répartition des responsabilités autour de la passerelle Azure	37
4.1	Technologies communes aux deux systèmes	40
4.2	Comparaison des principaux incréments Java/Spring Boot	42
4.3	Fiche de l'agent d'analyse	44
4.4	Fiche de l'agent de révision de l'analyse	45
4.5	Fiche de l'agent de planification	45
4.6	Fiche de l'agent de révision du plan	46
4.7	Fiche de l'agent de transformation	46
4.8	Fiche du proposeur de réparation	47
4.9	Fiche de l'agent de révision de la réparation	47
4.10	Fiche de l'assistant de migration	48
4.11	Principales difficultés et corrections	53
4.12	Configuration Azure OpenAI externalisée dans le système Java	54
5.1	Critères d'évaluation et disponibilité des mesures	58
5.2	Jalons de démonstration Java et Angular	58
5.3	Lien entre observations, décisions et évolutions du produit	59
5.4	Résultats d'une campagne d'intégration Java	60
5.5	Analyse qualitative selon des caractéristiques d'ISO/IEC 25010	63
5.6	Critères d'évaluation de l'intégration Azure OpenAI	66
5.7	Comparaison qualitative avec le processus manuel	67

Liste des acronymes

Acronyme	Signification
API	<i>Application Programming Interface</i>
CLI	<i>Command-Line Interface</i>
CSV	<i>Comma-Separated Values</i>
DTO	<i>Data Transfer Object</i>
HTTP	<i>Hypertext Transfer Protocol</i>
IA	Intelligence artificielle
IDE	<i>Integrated Development Environment</i>
JDK	<i>Java Development Kit</i>
JSON	<i>JavaScript Object Notation</i>
JVM	<i>Java Virtual Machine</i>
LLM	<i>Large Language Model</i>
LTS	<i>Long-Term Support</i>
MCP	<i>Model Context Protocol</i>
PFE	Projet de fin d'études
REST	<i>Representational State Transfer</i>
SSE	<i>Server-Sent Events</i>
SQL	<i>Structured Query Language</i>
UI	<i>User Interface</i>
UML	<i>Unified Modeling Language</i>
UoW	<i>Unit of Work</i>
UX	<i>User Experience</i>
ZIP	Format d'archivage compressé utilisé pour transporter les projets sources

Introduction générale

Les systèmes logiciels constituent un patrimoine stratégique pour les organisations, mais les langages, frameworks, bibliothèques et outils évoluent continuellement. Les applications reposant sur des versions anciennes deviennent alors plus difficiles à maintenir, sécuriser, tester et déployer. Leur migration exige de comprendre l'existant, de définir une trajectoire compatible, d'adapter les dépendances, la configuration et le code source, puis d'exécuter les installations, les compilations et les tests. Chaque échec peut ouvrir un nouveau cycle de diagnostic, de correction et de validation.

Les outils déterministes automatisent efficacement les transformations connues et les contrôles reproductibles, mais ils restent limités lorsque la résolution dépend fortement du contexte d'une application. À l'inverse, les grands modèles de langage peuvent analyser du code, interpréter des journaux et proposer des adaptations, sans offrir à eux seuls les garanties nécessaires pour contrôler des écritures, des commandes ou des décisions sensibles. Le problème n'est donc pas de choisir entre automatisation déterministe et intelligence générative, mais de les combiner dans un cadre maîtrisé.

Ce projet de fin d'études, réalisé au sein de **CGI Technologies et Solutions Maroc**, porte sur la conception et le développement d'une **Migration Factory intelligente, extensible et gouvernée**. La solution adopte une architecture multi-agents hybride et centralement orchestrée. Les agents sont spécialisés dans les activités de raisonnement, de révision, de planification, de transformation agentique, de réparation et d'assistance. Les services déterministes conservent l'autorité sur l'état du workflow, les transitions, les profils, les chemins, les commandes, les écritures et les validations. Le développeur reste responsable des décisions qui engagent la suite du processus.

La démarche est appliquée à deux écosystèmes. La déclinaison Java/Spring Boot couvre une trajectoire progressive entre Spring Boot 2.1, 2.7, 3.5 et 4.0 avec plusieurs versions de Java. Elle constitue l'implémentation la plus mature et intègre notamment les espaces de travail isolés, les validations Maven, la réparation gouvernée, le cockpit, l'assistant et le reporting. Les appels aux modèles y sont configurés au moyen d'Azure OpenAI, avec des déploiements externalisés selon les rôles agentiques. La déclinaison Angular permet d'éprouver les mêmes principes dans un second écosystème ; les preuves retenues valident les transitions 18 vers 19 et 19 vers 20, tandis que le parcours élargi 11 vers 21 reste à compléter.

L'utilisateur principal est le développeur chargé de la migration. L'objectif n'est pas de supprimer son intervention, mais de réduire les opérations répétitives, de structurer le diagnostic et de lui fournir un environnement où l'état, les preuves, les blocages et les actions autorisées sont explicitement visibles.

La problématique générale peut être formulée comme suit :

Comment concevoir et mettre en œuvre une architecture multi-agents hybride capable de coordonner l'analyse, la planification, la transformation,

la validation et la réparation de migrations Java/Spring Boot et Angular, tout en préservant l'intégrité du code source, la traçabilité des décisions, la reproductibilité des opérations et le contrôle du développeur ?

Pour y répondre, le projet poursuit plusieurs objectifs complémentaires : identifier les agents nécessaires, distinguer les services déterministes, définir leurs responsabilités et leurs limites d'autorité, concevoir l'orchestrateur et les contrats d'échange, intégrer les outils propres à chaque écosystème, mettre en œuvre une réparation gouvernée et évaluer les résultats obtenus.

Sauf indication contraire, l'état présenté dans ce rapport est mis à jour au **30 août 2026**. Une étape n'est considérée comme validée que lorsqu'elle est appuyée par une preuve technique ou un élément de validation explicitement intégré au rapport ; les objectifs encore ouverts sont présentés comme tels.

Le rapport est organisé en cinq chapitres :

- le **Chapitre I** présente le contexte, les besoins, les objectifs et la méthodologie ;
- le **Chapitre II** expose le cadrage conceptuel, l'état de l'art et le positionnement de la solution ;
- le **Chapitre III** décrit la conception de l'architecture multi-agents et de sa gouvernance ;
- le **Chapitre IV** présente l'architecture technique et l'implémentation des deux déclinaisons ;
- le **Chapitre V** formalise le protocole d'évaluation, les résultats, l'analyse comparative et les limites.

La conclusion générale synthétise les apports du projet, ses limites actuelles et les perspectives d'industrialisation.

Contexte général du projet, analyse des besoins et spécification du système

Introduction

Ce chapitre situe le projet dans son environnement professionnel et précise le besoin auquel il répond. Il présente l'organisme d'accueil, le domaine de la modernisation applicative, les utilisateurs visés, les catégories de composants mobilisées, la genèse du produit, la problématique, les exigences, les objectifs et la méthodologie suivie.

La spécification s'appuie sur les principes d'ISO/IEC/IEEE 29148 : une exigence doit être compréhensible, reliée à une partie prenante et associée à un moyen de vérification [1]. Le rapport distingue donc l'objectif du produit, l'état observé dans les deux prototypes et les évolutions d'industrialisation. L'état d'avancement décrit dans cette version est mis à jour au **30 août 2026**.

1.1 Présentation de l'organisme d'accueil

1.1.1 Le groupe CGI

Fondée en 1976 au Québec, CGI figure parmi les principales entreprises mondiales de conseil en technologies de l'information et en management. À l'occasion de son cinquantième anniversaire en 2026, le groupe indique compter environ 94 000 professionnels, être présent dans plus de 400 implantations et avoir réalisé un chiffre d'affaires de 15,9 milliards de dollars canadiens pour l'exercice 2025 [21, 22].



Figure 1.1 – Logo de CGI

L'offre de CGI couvre notamment le conseil, l'intégration de systèmes, le développement et la maintenance d'applications, les services managés, le cloud, la cybersécurité, les données et l'intelligence artificielle. La modernisation des applications s'inscrit naturellement dans ces activités, car elle vise à prolonger la durée de vie des systèmes et à réduire leur dette technique.

1.1.2 CGI Technologies et Solutions Maroc

CGI dispose au Maroc de centres implantés à Rabat, Casablanca et Fès. Le site officiel de l'entreprise indique que ces centres regroupent plus de 1 300 professionnels et fournissent des services applicatifs, d'infrastructure et de support sur des technologies modernes et historiques [23]. Le stage s'est déroulé au centre de Rabat, situé à Technopolis, Sala Al Jadida.

Le projet est une initiative interne. L'équipe opérationnelle comprend trois développeurs, accompagnés par deux référents fonctionnels, un expert technique et les responsables intervenant lors des revues et démonstrations.

Partie prenante	Contribution au projet
Équipe de développement	Analyse, conception, implémentation, intégration, tests et expérimentation.
Référents fonctionnels	Clarification des attentes, suivi du backlog et retour sur les incréments.
Expert technique	Examen des orientations, des risques et de la cohérence de l'architecture.
Responsables CGI	Priorisation, suivi de l'avancement et appréciation des démonstrations.

Table 1.1 – Parties prenantes du projet

1.2 Présentation du projet

1.2.1 Domaine et finalité

Le projet relève de la modernisation logicielle, de l'intelligence artificielle appliquée au génie logiciel et de l'orchestration de systèmes multi-agents. La solution, appelée **Migration Factory**, vise à conduire des migrations progressives, contrôlées et traçables en combinant :

- des **agents intelligents** pour les tâches nécessitant interprétation, raisonnement, planification, révision ou assistance ;
- des **services déterministes** pour les états, transitions, commandes, chemins, écritures, validations et preuves ;
- des **outils d'exécution** propres aux écosystèmes, tels que Maven, OpenRewrite, npm et Angular CLI ;
- une **supervision humaine** aux points où une décision peut engager la suite du processus ou modifier le code.

La première déclinaison concerne Java/Spring Boot. La seconde concerne Angular. Elles reposent sur les mêmes principes de gouvernance, mais sont développées dans deux dépôts indépendants afin de préserver leurs runtimes, leurs outils et leurs trajectoires de versions.

1.2.2 Utilisateurs cibles

L'utilisateur opérationnel actuel est le **développeur chargé de la migration**. Il fournit l'application, configure les versions, contrôle les prérequis, lance l'exécution, examine les preuves et prend les décisions nécessaires.

Dans une version industrialisée, d'autres rôles pourraient être distingués : agent de révision technique, responsable de migration, administrateur de plateforme et lecteur de rapports. Ces rôles constituent une cible d'évolution ; ils ne sont pas implémentés dans les prototypes locaux actuels, qui ne possèdent ni comptes, ni authentification, ni contrôle d'accès par rôle.

1.2.3 Classification des acteurs et composants

La figure 1.2 sépare les catégories afin de ne pas transformer artificiellement tout composant du workflow en agent.

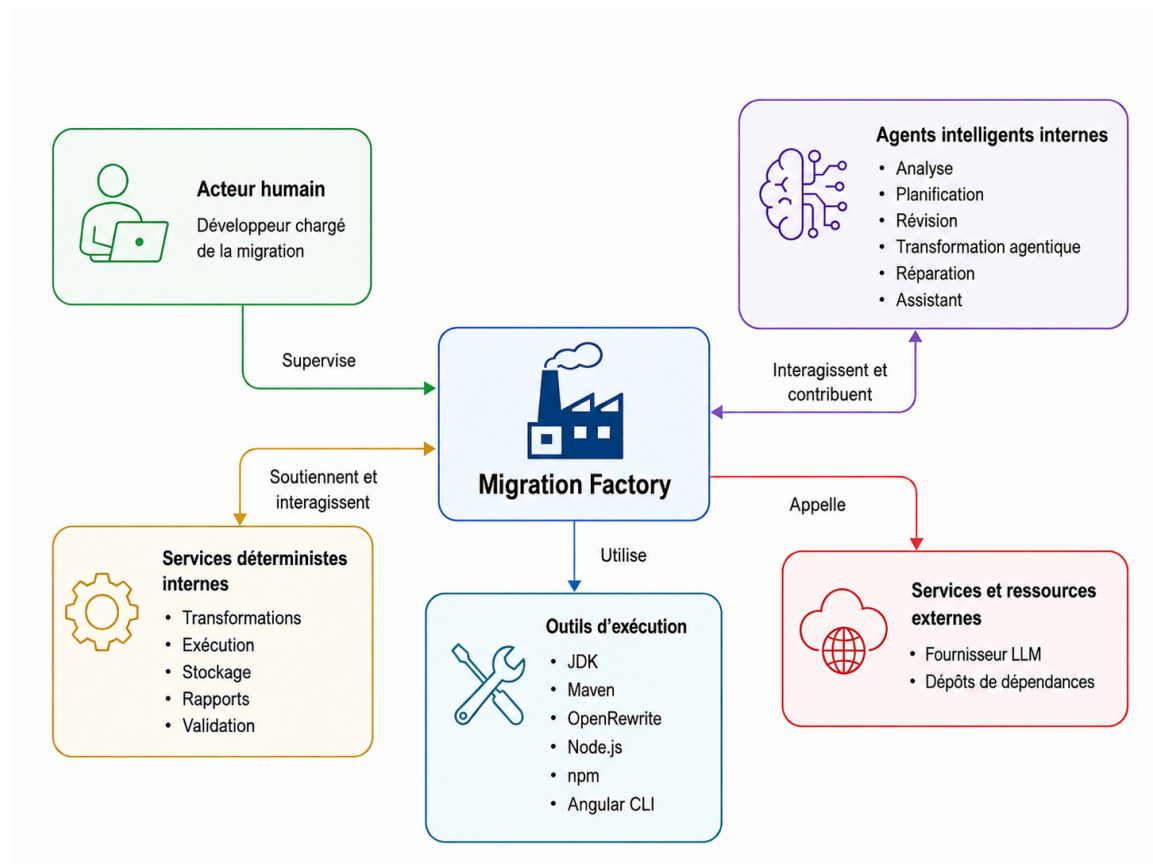


Figure 1.2 – Vue de contexte et classification des acteurs et composants

Le système de fichiers local appartient à l'infrastructure d'exécution ; il n'est pas considéré comme un service externe. Les agents n'accèdent pas librement au terminal ou aux fichiers : les services déterministes médiatisent les commandes, les écritures et les transitions.

1.2.4 Fonctionnalités existantes

Le système Java/Spring Boot est le plus mature. Il prend en charge une trajectoire progressive Spring Boot 2.1 → 2.7 → 3.5 → 4.0, avec Java 11, 17 et 21 selon les étapes. Il comprend la qualification, l'analyse, la planification, les décisions humaines, les espaces de travail isolés, les transformations, Maven, les tests, une boucle de réparation, un cockpit, un assistant et un rapport final.

Le système Angular est en développement actif sur une trajectoire progressive visant Angular 11 vers Angular 21. Les agents principaux d'analyse, de révision, de planification et de transformation sont développés. La preuve d'exécution versionnée actuellement retenue valide les transitions Angular 18 vers 19 et 19 vers 20 ; le stage 20 vers 21 est préparé mais n'est pas encore démarré. Cette preuve confirme le fonctionnement du mécanisme par stages sur le périmètre exercé, sans démontrer encore la trajectoire complète 11 vers 21.

1.2.5 Genèse et évolution générale du projet

Le produit n'a pas été construit comme une architecture complète dès le premier jour. Il a évolué par incréments successifs, chacun répondant à un besoin observé ou à une limite de la version précédente.

La première phase, menée pendant environ trois semaines à partir du début du mois de mai 2026, a regroupé l'analyse du besoin, l'étude des trajectoires Spring Boot, la préparation des environnements et le développement du premier workflow en ligne de commande. Cette preuve de concept a été présentée le 25 mai 2026. Une deuxième version CLI, corrigée et enrichie, a ensuite été présentée le 3 juin à une audience CGI plus large. Le 19 juin, la troisième démonstration a marqué le passage à une plateforme full stack comprenant backend, frontend, cockpit et chatbot.

À partir du 20 juin, les travaux ont porté sur la consolidation d'une V1 : nouveau frontend, amélioration du travail des agents LLM, reporting enrichi et boucle de réparation gouvernée. La déclinaison Angular a commencé à la mi-juillet 2026 avec un premier périmètre 18–21, puis l'objectif a été étendu à une trajectoire 11–21. La preuve d'exécution versionnée intégrée au rapport établit deux stages scellés, 18 vers 19 puis 19 vers 20 ; les étapes complémentaires restent à démontrer avant de présenter la couverture 11–21 comme acquise.

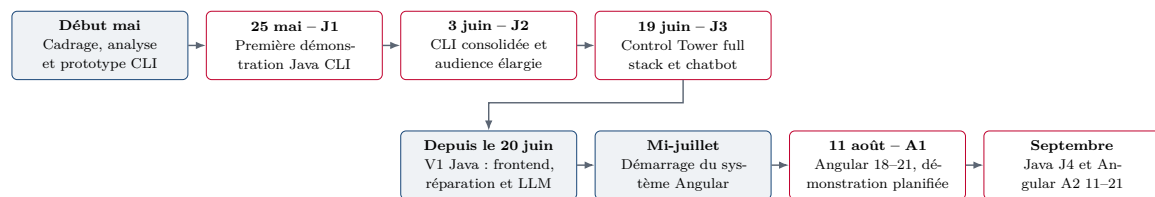


Figure 1.3 – Frise chronologique générale du projet

1.3 Problématique traitée

1.3.1 Constats et besoin de coordination

Une migration combine inventaire, détection de versions, résolution de compatibilités, planification, transformation, installation, compilation, tests, diagnostic, réparation et production de preuves. La difficulté provient autant de ces tâches que de leur coordination : choisir le bon traitement, transmettre le contexte utile, préserver l'ordre, suspendre à un point de validation, reprendre après décision et empêcher l'application d'un résultat obsolète.



Figure 1.4 – Dépendances et boucles de coordination d'une migration

1.3.2 Limites d'une architecture monolithique ou d'un agent unique

Un agent unique devrait simultanément comprendre le projet, planifier, transformer, exécuter, valider et se corriger. Cette concentration entraîne un contexte trop large, une séparation insuffisante entre proposition et autorité, une faible testabilité et une propagation accrue des erreurs. Elle complique également l'adaptation à deux écosystèmes différents.

Le projet retient donc une architecture multi-agents hybride, spécialisée et centralement orchestrée. Les agents possèdent des objectifs bornés ; l'orchestrateur organise leur séquence ; les services déterministes conservent l'autorité sur l'exécution.

1.3.3 Problématique générale

Comment concevoir et mettre en œuvre une architecture multi-agents hybride capable de coordonner l'analyse, la planification, la transformation, la validation et la réparation de migrations Java/Spring Boot et Angular, tout en préservant l'intégrité du code source, la traçabilité des décisions, la reproductibilité des opérations et le contrôle du développeur ?

1.4 Besoins fonctionnels

Table 1.2 – Besoins fonctionnels et maturité des deux systèmes

ID	Besoin	Java/Spring Boot	Angular
BF-01	Configurer la source, la cible et les profils d'exécution.	Implémenté	En intégration
BF-02	Vérifier chemins, runtimes, outils et compatibilités.	Implémenté	En validation

ID	Besoin	Java/Spring Boot	Angular
BF-03	Analyser structure, dépendances, configuration, tests et risques.	Intégré	Agent implémenté ; intégration en cours
BF-04	Construire une trajectoire progressive et un plan révisé.	Intégré	Agents implémentés
BF-05	Approuver, rejeter ou demander une révision.	Avancé	En intégration
BF-06	Créer un espace isolé sans modifier la source.	Validé	Implémenté ; validation globale en cours
BF-07	Appliquer les transformations propres à l'écosystème.	Avancé	Implémenté ; essais en cours
BF-08	Installer, compiler ou construire et exécuter les tests.	Intégré	En validation
BF-09	Conserver les échecs et construire un contexte de diagnostic.	Implémenté	À intégrer
BF-10	Proposer, réviser, approuver et revalider une réparation.	Implémenté	Planifié
BF-11	Afficher étapes, événements, preuves, décisions et blocages.	Avancé	En intégration
BF-12	Annuler ou reprendre selon les transitions autorisées.	Partiel	En développement
BF-13	Produire un rapport final relié aux preuves.	Implémenté	En développement
BF-14	Gérer identités, rôles et autorisations.	Non implémenté	Non implémenté

1.5 Besoins non fonctionnels

Table 1.3 – Exigences non fonctionnelles et critères de vérification

ID	Caractéristique	Exigence	Critère de vérification
BNF-01	Intégrité	La source externe ne doit pas être modifiée.	Comparer son empreinte avant et après l'exécution.
BNF-02	Isolation	Les écritures doivent rester dans l'espace autorisé.	Refuser tout chemin sortant du répertoire de l'exécution.
BNF-03	Traçabilité	Chaque action doit être rattachée à une migration.	Vérifier identifiant, horodatage, producteur et référence de preuve.
BNF-04	Reproductibilité	Les paramètres et commandes essentiels doivent être persistés.	Reconstituer le profil et le manifeste d'une commande exécutée.
BNF-05	Fiabilité	Une étape ne progresse qu'après validation.	Contrôler préconditions, code de sortie et résultats de tests.
BNF-06	Reprise	Une interruption ne doit pas supprimer l'état validé.	Reprendre depuis un état persistant autorisé.
BNF-07	Idempotence	Une requête protégée ne doit pas être appliquée deux fois.	Répéter la même clé et vérifier l'absence d'effet supplémentaire.
BNF-08	Confidentialité	Le contexte LLM doit exclure ou masquer les secrets connus.	Contrôler les règles d'exclusion et les extraits transmis.
BNF-09	Gouvernance	Un agent ne doit pas appliquer seul un changement sensible.	Vérifier l'existence des contrôles déterministes et de la décision humaine.
BNF-10	Maintenabilité	Agents, services et profils doivent rester séparables.	Remplacer un agent ou un profil sans modifier l'ensemble du workflow.
BNF-11	Utilisabilité	Le cockpit doit montrer l'état, le blocage et l'action suivante.	Vérifier ces informations dans la projection affichée.
BNF-12	Évolutivité	De nouveaux profils doivent être ajoutables sans refonte globale.	Ajouter un profil versionné et vérifier sa résolution isolée.

1.6 Objectifs du projet

1.6.1 Objectif général

L'objectif général consiste à concevoir, implémenter et évaluer une architecture multi-agents gouvernée permettant d'automatiser une part importante des migrations logicielles, tout en maintenant des validations déterministes et une supervision humaine.

1.6.2 Objectifs spécifiques

Afin d'atteindre l'objectif général, les objectifs spécifiques sont formulés comme des résultats que le projet doit produire ou rendre possibles. Ils sont présentés dans l'ordre logique de construction de la solution :

- Identifier les rôles agentiques nécessaires et distinguer les responsabilités qui doivent rester confiées aux services déterministes ;
- Définir, pour chaque agent, son objectif, ses entrées, ses traitements, ses sorties, son niveau d'autonomie et ses limites d'autorité ;
- Concevoir une architecture SMA hybride, spécialisée et centralement orchestrée, compatible avec une supervision humaine ;
- Formaliser les contrats de communication, les artefacts échangés et les règles de traçabilité entre agents, orchestrateur et services ;
- Définir les mécanismes de gestion du cycle de vie d'une migration, notamment les états, transitions, interruptions, reprises et replanifications ;
- Implémenter l'architecture SMA dans les écosystèmes Java/Spring Boot et Angular en respectant les contraintes propres à leurs runtimes et outils ;
- Mettre en œuvre une boucle de réparation gouvernée intégrant proposition, révision, décision humaine, application contrôlée et revalidation ;
- Mettre en place un cockpit de supervision permettant de suivre les étapes, les preuves, les blocages, les décisions et les actions autorisées ;
- Valider le fonctionnement de la solution à partir de tests, de scénarios de migration, de démonstrations et des métriques effectivement disponibles ;
- Préparer l'extensibilité de la Migration Factory vers de nouvelles versions, de nouveaux profils de migration et d'autres écosystèmes.

1.7 Méthodologie générale

1.7.1 Organisation Agile du projet

Le projet a été conduit selon une démarche Agile inspirée de Scrum, avec des sprints de deux semaines, des points quotidiens, des revues ou démonstrations et une rétrospective interne [14]. Jira est utilisé pour le backlog, Microsoft Teams pour les échanges, GitHub pour le versionnement et les revues, et Confluence pour la documentation.

Le cycle de travail associe backlog, sélection d'un périmètre, conception, implémentation, tests, démonstration et adaptation. Les présentations réalisées devant les parties prenantes sont qualifiées dans ce rapport de **jalons de démonstration et de validation**. Elles ne sont assimilées à une Sprint Review que lorsque l'équipe et les parties prenantes ont effectivement inspecté le résultat et ajusté la suite du backlog.

1.7.2 Planification prévisionnelle et rétrospective

La conduite Agile du projet n'empêche pas de représenter les grandes périodes de travail et les jalons attendus. Deux vues sont distinguées ci-dessous. La première est une **reconstitution prévisionnelle** construite à partir des jalons initialement fixés et des grandes phases documentées dans le backlog ; aucun diagramme de Gantt prévisionnel original n'a été retrouvé parmi les artefacts de suivi consultés. La seconde est une vue **rétrospective**, qui décrit l'enchaînement effectivement observé et les objectifs encore planifiés à la date de référence du rapport. Cette distinction évite de présenter a posteriori le déroulement réel comme s'il avait été entièrement prévu dès le départ.

Planification prévisionnelle reconstituée. La vue suivante représente le cadrage macroscopique attendu : valider d'abord la faisabilité Java, consolider progressivement la plateforme, puis adapter les mêmes principes à Angular avant les jalons de septembre.

Activité ou jalon prévu	Mai	Juin	Juil.	Août	Sept.	Nature
Cadrage, analyse et preuve de concept Java						Phase Java
Première démonstration Java	◆					Jalon
Stabilisation du pipeline Java et validation élargie						Phase Java
Passage à une plateforme backend/frontend et démonstration						Phase Java / jalon
Consolidation de la V1 Java, réparation et amélioration LLM						Phase Java
Étude puis premier périmètre Angular 18–21						Phase Angular
Première démonstration Angular				◆		Jalon
Extension progressive Angular 11–21						Phase Angular
Jalons Java V1 et Angular 11–21					◆	Jalon
Rédaction et validation du rapport						Documentation

Table 1.4 – Diagramme de Gantt prévisionnel reconstitué du projet Migration Factory

Activités Java
 Activités Angular
 Rédaction
 ◆ Jalon

Planification rétrospective. Le diagramme suivant décrit l'évolution réellement observée ou, pour les éléments encore futurs à la date de référence du rapport, les objectifs explicitement planifiés. Cette représentation macroscopique est compatible avec une conduite itérative ; ISO 21502 reconnaît les approches prédictives, incrémentales, itératives, adaptatives et hybrides, y compris les approches agiles [6].

Activité ou jalon	Mai	Juin	Juil.	Août	Sept.	Statut
Cadrage, étude et environnement Java						Réalisé
Prototype Java CLI						Réalisé
Java J1 – 25 mai	◆					Réalisé
Consolidation CLI et Java J2 – 3 juin		◆				Réalisé
Backend, frontend et chatbot						Réalisé
Java J3 – 19 juin		◆				Réalisé
Nouveau frontend et consolidation V1						En cours
Réparation gouvernée et amélioration LLM						En cours
Étude et agents Angular						En cours
Angular A1 – 11 août, périmètre 18–21				◆		Planifié
Extension Angular 11–21						Planifié
Java J4 et Angular A2					◆	Planifié
Rédaction, validation et soutenance						En cours

Table 1.5 – Diagramme de Gantt rétrospectif du projet Migration Factory

Activités Java
 Activités Angular
 Rédaction
 ◆ Jalon

1.7.3 Itérations, incréments et jalons de validation

Cette partie complète la lecture temporelle du Gantt en montrant **ce que chaque période de travail devait produire**. Le tableau relie ainsi quatre éléments : la période considérée, l'objectif poursuivi, l'incrément effectivement obtenu et le jalon servant de point de démonstration ou de validation. Il permet de suivre la progression du projet non seulement par dates, mais aussi par résultats fonctionnels et techniques successifs.

Table 1.6 – Synthèse des principales itérations et des incréments

Période	Objectif	Incrément produit	Jalon
Début mai–24 mai	Cadrer le besoin et valider la faisabilité technique.	Environnement, trajectoire Java et première chaîne de migration CLI.	Préparation J1
25 mai	Présenter la première version opérationnelle.	Analyse, planification, transformation, Maven et artefacts en CLI.	Java J1 réalisé

Période	Objectif	Incrément produit	Jalon
26 mai–2 juin	Corriger et stabiliser le prototype.	Workflow CLI enrichi, sorties améliorées et scénario consolidé.	Préparation J2
3 juin	Élargir la validation auprès de CGI.	Version CLI améliorée présentée à davantage de responsables et directeurs.	Java J2 réalisé
4–18 juin	Passer du prototype à une plateforme intégrée.	Backend FastAPI, SQLite, frontend, cockpit, événements et chatbot.	Préparation J3
19 juin	Présenter la Control Tower full stack.	Plateforme intégrée avec backend, frontend et assistant.	Java J3 réalisé
Depuis le 20 juin	Consolider une V1 gouvernée.	Nouveau frontend, réparation proposeur/réviseur, reporting et amélioration LLM.	Java J4 planifié
Depuis mi-juillet	Adapter les principes à Angular.	Migration manuelle d'étude, agents et workflow Angular 18–21.	Angular A1 planifié
Août–septembre	Étendre le périmètre Angular.	Trajectoire progressive d'Angular 11 vers Angular 21.	Angular A2 planifié

Les numéros détaillés des sprints et les tickets associés restent disponibles dans Jira. Le rapport privilégie une synthèse des incréments plutôt qu'une reproduction exhaustive des tableaux de suivi.

1.7.4 Cycle technique

La progression technique suit une boucle courte d'apprentissage et de validation. Chaque incrément part d'un problème observé, le transforme en exigence, puis en solution conçue et implémentée. La vérification technique et la démonstration produisent ensuite de nouvelles observations qui alimentent l'itération suivante.

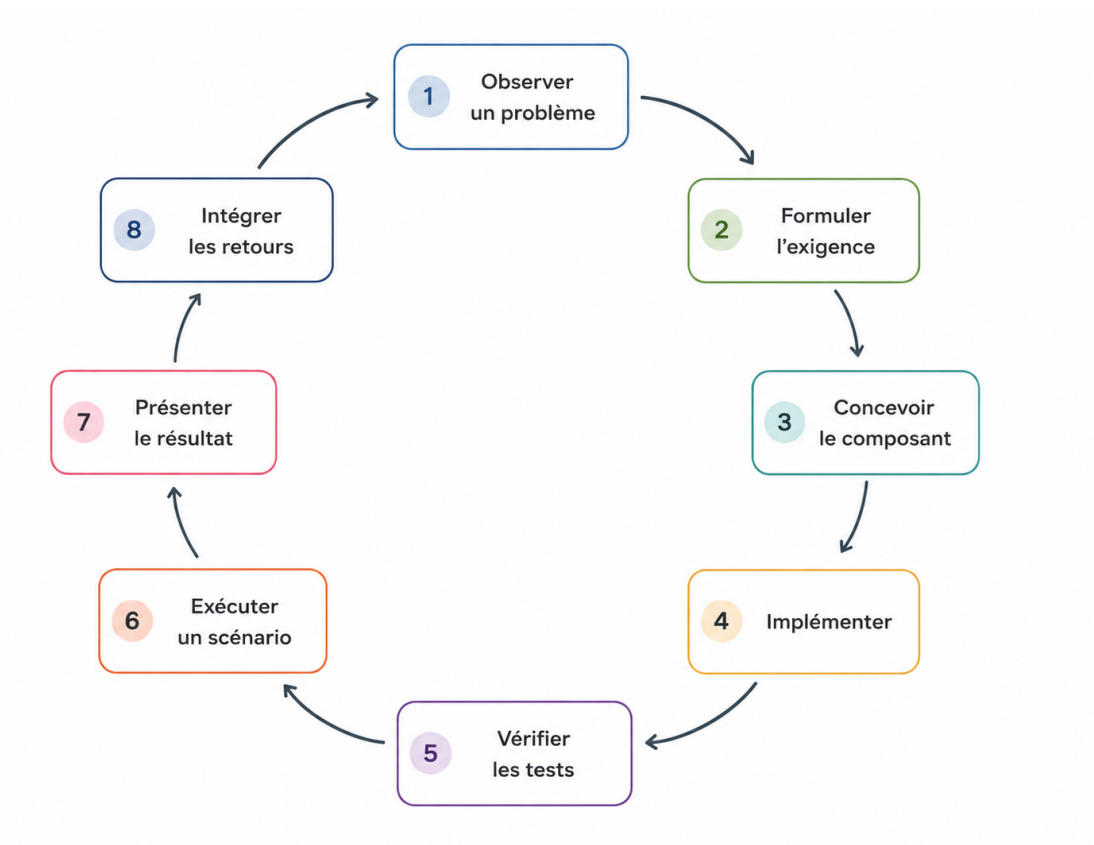


Figure 1.5 – Cycle itératif de conception, réalisation et validation

Java/Spring Boot a servi de premier terrain de validation de cette boucle. Les limites observées sur les versions successives ont orienté les incréments suivants, tandis qu'Angular permet d'éprouver la transférabilité du modèle dans un second écosystème sans imposer une base de code d'exécution commune.

1.8 Contraintes d'intégration des modèles dans l'environnement CGI

Le choix du fournisseur de modèles ne constitue pas uniquement une décision d'implémentation. Il répond également à une contrainte d'intégration dans l'environnement technologique du client. CGI utilisant principalement les services Microsoft Azure, la déclinaison Java/Spring Boot de la Migration Factory s'appuie sur Azure OpenAI pour l'accès aux grands modèles de langage.

L'objectif n'est pas d'imposer un modèle unique géré ou embarqué par la plateforme. La solution doit permettre à l'organisation de connecter les modèles qu'elle a elle-même déployés et gouvernés dans sa ressource Azure. Le backend reçoit alors une configuration externalisée comprenant l'endpoint de la ressource, le mécanisme d'authentification et les noms des déploiements associés aux rôles agentiques. Azure distingue en effet la ressource, son endpoint et les déploiements nommés appelés par les applications [17].

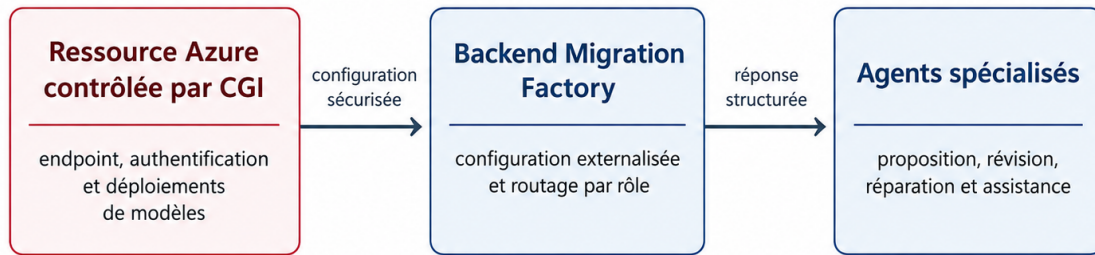


Figure 1.6 – Principe d'intégration des déploiements Azure contrôlés par CGI

Cette orientation correspond à une logique d'*apport de ses propres déploiements Azure* : l'organisation ne téléverse pas nécessairement un modèle dans la Migration Factory ; elle fournit les références nécessaires pour consommer des modèles déjà déployés dans son environnement.

Les exigences suivantes traduisent cette orientation en résultats attendus et en critères vérifiables :

ID	Nature	Exigence	Critère de vérification
BF-15	Fonctionnelle	Permettre à CGI de connecter ses propres déploiements de modèles Azure sans modifier le code applicatif.	Changer le nom d'un déploiement par configuration et vérifier que le rôle correspondant utilise la nouvelle valeur.
BNF-13	Configurabilité	L'endpoint, l'authentification et les déploiements doivent être externalisés.	Vérifier qu'aucune valeur propre à une ressource cliente n'est codée en dur dans les agents.
BNF-14	Confidentialité	Les clés, endpoints complets et identifiants sensibles ne doivent pas être exposés par l'interface ou les projections publiques.	Inspecter les réponses API, journaux et artefacts publics et vérifier le masquage des valeurs sensibles.

Table 1.7 – Exigences liées à l'intégration Azure OpenAI

Le code de la déclinaison Java confirme actuellement cette orientation. Pour Angular, Azure constitue une cible cohérente avec l'environnement client, mais son utilisation effective doit rester distinguée de l'intégration Java tant qu'elle n'est pas confirmée par une version audité du dépôt Angular.

Conclusion

Ce chapitre a présenté le contexte professionnel, les utilisateurs, les catégories de composants, la problématique et les besoins de la Migration Factory. La distinction entre agents intelligents, services déterministes, outils d'exécution et services externes clarifie les frontières du système. Les exigences sont associées à la maturité de chaque dépôt et, pour les qualités critiques, à un critère de vérification.

La chronologie, le Gantt rétrospectif et les jalons montrent que l'architecture résulte d'une progression expérimentale : première faisabilité CLI, consolidation, transformation en plateforme full stack, préparation d'une V1 gouvernée et extension à Angular. Le chapitre suivant présente les fondements théoriques relatifs aux agents, aux systèmes multi-agents, à l'orchestration et à la migration, puis compare les travaux et plateformes proches du projet.

Cadrage conceptuel et état de l'art

Introduction

La Migration Factory combine agents intelligents, orchestration, services déterministes, outils de migration et supervision humaine. Sa compréhension exige donc un cadrage portant à la fois sur les agents, les systèmes multi-agents, l'orchestration et la migration logicielle.

Le chapitre présente d'abord les notions théoriques sous une forme pédagogique, puis analyse un corpus de travaux et de plateformes couvrant la migration déterministe, la modernisation assistée par IA, la réparation automatique et l'orchestration multi-agents. La comparaison prend en compte le domaine, les agents, le mode d'orchestration, les métriques disponibles et les limites.

2.1 Agents intelligents

2.1.1 Définition et interaction avec l'environnement

Un agent logiciel est une entité située dans un environnement, capable de recevoir des perceptions, de maintenir un état ou un contexte, puis d'agir afin de poursuivre un objectif. Wooldridge et Jennings distinguent la théorie des agents, leurs architectures et les langages utilisés pour les construire [8].

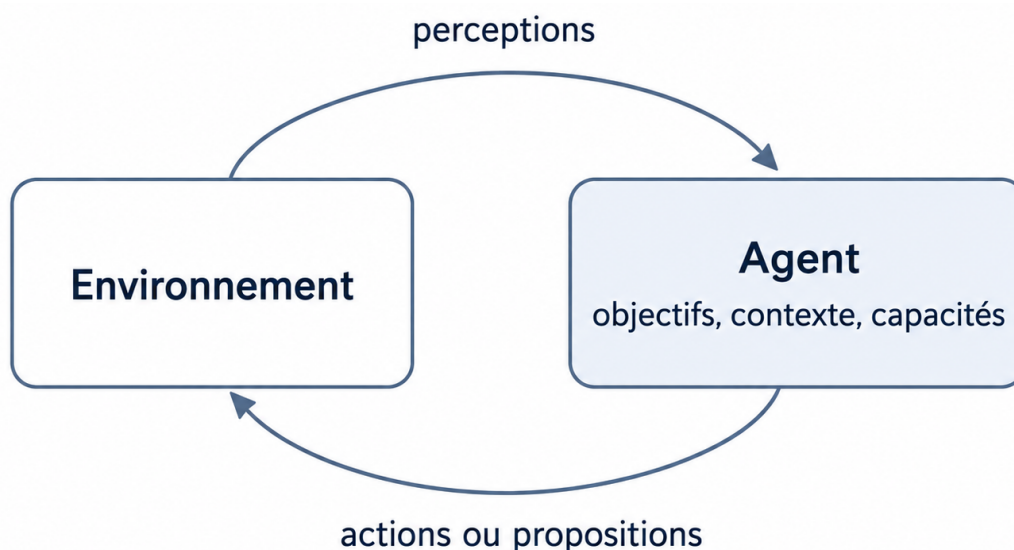


Figure 2.1 – Interaction générale entre un agent et son environnement

Dans ce projet, les agents sont spécialisés et bornés. Ils reçoivent un contexte préparé, produisent une sortie structurée et rendent la main à l'orchestrateur. Ils ne disposent pas d'un accès libre au terminal ou au système de fichiers.

2.1.2 Propriétés

La littérature associe aux agents plusieurs propriétés [9, 10] :

- **autonomie** : choix local d'un traitement sans intervention continue ;
- **réactivité** : réponse aux changements observés ;
- **proactivité** : poursuite d'un objectif ;
- **sociabilité** : interaction avec d'autres agents ou acteurs ;
- **raisonnement** : sélection d'une réponse à partir du contexte ;
- **adaptabilité** : ajustement à de nouvelles preuves ou contraintes.

Ces propriétés existent à des degrés différents. L'autonomie de raisonnement d'un agent peut être importante alors que son autonomie d'action reste nulle. Cette distinction est essentielle dans un système gouverné.

2.1.3 Types d'agents

Type	Rôle général	Correspondance dans le projet
Agent cognitif	Raisonne sur plusieurs informations et contraintes.	Analyseur et proposeur de réparation.
Agent de planification	Décompose un objectif en tâches ordonnées.	Planificateur de trajectoire et de stage.
Agent évaluateur	Examine la qualité ou la conformité d'une sortie.	Agents de révision de l'analyse, du plan et de la réparation.
Agent conversationnel	Interagit en langage naturel avec l'utilisateur.	Assistant de migration.
Agent de transformation	Produit des changements candidats à partir d'un plan.	Transformateur Java ou Angular lorsque le traitement est agentique.
Agent réactif	Répond à un événement précis.	Peut être utilisé pour une classification intelligente ; les contrôles purement déterministes restent des services.

Table 2.1 – Types d'agents et correspondance avec la Migration Factory

2.1.4 Agent, LLM, service et outil

Un **LLM** produit une sortie probabiliste à partir d'un contexte. Un **agent** associe un modèle éventuel à un rôle, des objectifs, des règles d'interaction et un schéma de sortie. Un **service déterministe** applique une capacité vérifiable par contrat. Un **outil** réalise une opération technique, par exemple Maven ou Angular CLI.

Ainsi, le résolveur de profils, l'exécuteur de commandes, le service de transitions, le stockage d'artefacts et la génération de rapports ne sont pas qualifiés d'agents dans ce rapport.

2.2 Systèmes multi-agents

2.2.1 Définition

Un système multi-agents (SMA) regroupe plusieurs agents capables d'interagir dans un environnement partagé afin de contribuer à un objectif collectif. Les responsabilités, connaissances et actions sont réparties entre plusieurs entités [9, 10, 11].

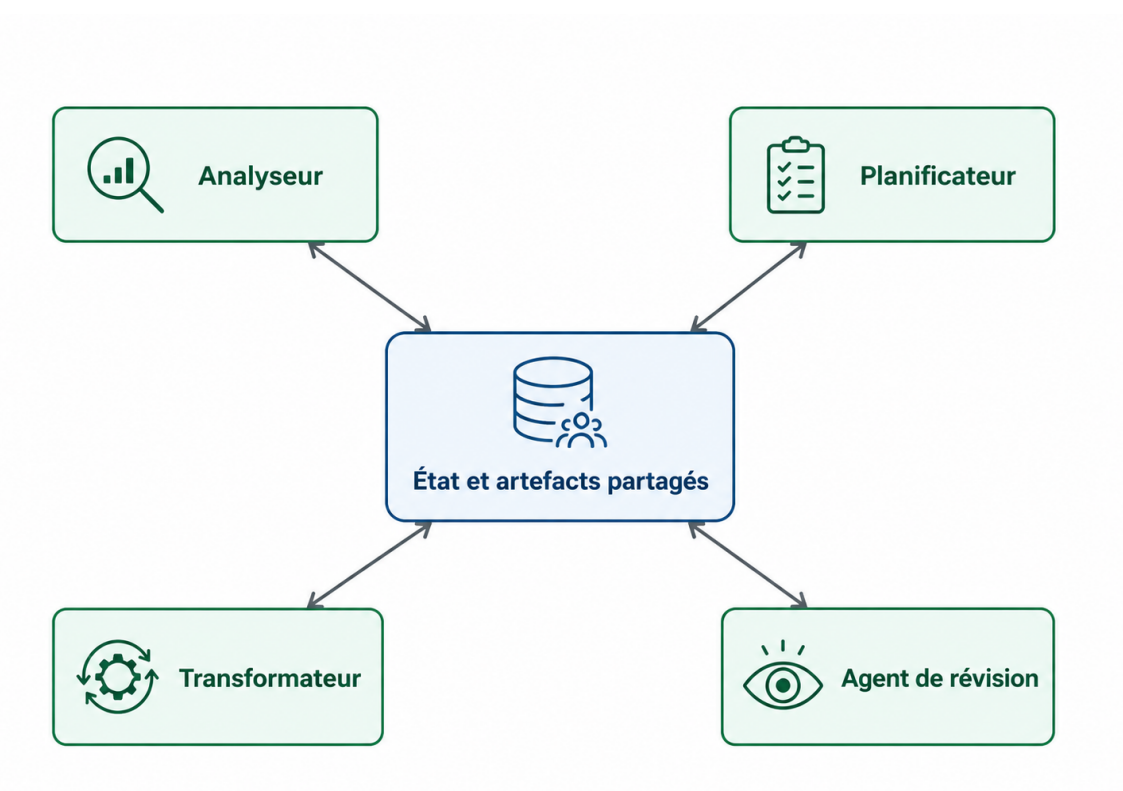


Figure 2.2 – Vue simplifiée d'un SMA spécialisé

2.2.2 Coordination, communication et délégation

La coordination respecte les dépendances entre actions. La communication transmet informations, demandes ou décisions. La délégation confie une sous-tâche à un agent compétent. La coopération associe plusieurs contributions à un même objectif. La négociation intervient lorsque des préférences ou ressources doivent être arbitrées.

FIPA définit des structures de messages, des actes communicatifs et des protocoles d'interaction entre agents [12]. La Migration Factory ne prétend pas implémenter FIPA-ACL ; elle reprend seulement le principe de messages explicites comportant tâche, contexte, statut, résultat et corrélation.

2.2.3 Avantages et limites

La spécialisation apporte modularité, testabilité, séparation des responsabilités et limitation du contexte. Elle permet également de remplacer un modèle ou une politique sans refondre tout le processus. En contrepartie, un SMA introduit davantage de contrats, d'états, de latence et de risques de désynchronisation. Le recours à plusieurs agents n'est donc justifié que lorsque les rôles sont réellement distincts [26].

2.3 Orchestration d'agents

2.3.1 Notions à distinguer

Notion	Définition	Application au projet
Orchestration	Un composant central choisit l'ordre, les transitions et les participants.	Modèle principal.
Chorégraphie	Les participants réagissent à un protocole partagé sans contrôleur unique.	Non retenue comme modèle principal [13].
Coordination	Mécanismes évitant conflits et incohérences.	États, points de validation, empreintes et idempotence.
Planification	Construction d'une séquence permettant d'atteindre une cible.	Trajectoire de versions et unités de migration.

Table 2.2 – Orchestration, chorégraphie, coordination et planification

2.3.2 Modèles d'organisation

Les modèles courants sont l'orchestrateur central, les agents distribués, l'organisation hiérarchique et l'architecture hybride. La Migration Factory adopte un modèle hybride : l'orchestrateur centralise l'état et les transitions, alors que les agents conservent

une autonomie locale de raisonnement. LangGraph fournit le graphe d'exécution, les checkpoints et les interruptions, mais ne constitue pas la source d'autorité métier [25].

2.4 Plateformes et architectures intelligentes

Une plateforme agentique associe interface, API, orchestrateur, agents, services déterministes, persistance, artefacts, observabilité et sécurité. Les agents peuvent être intégrés comme modules internes ou services séparés. Une architecture microservices n'est pas une condition nécessaire : le critère central reste la séparation explicite des responsabilités et des contrats.

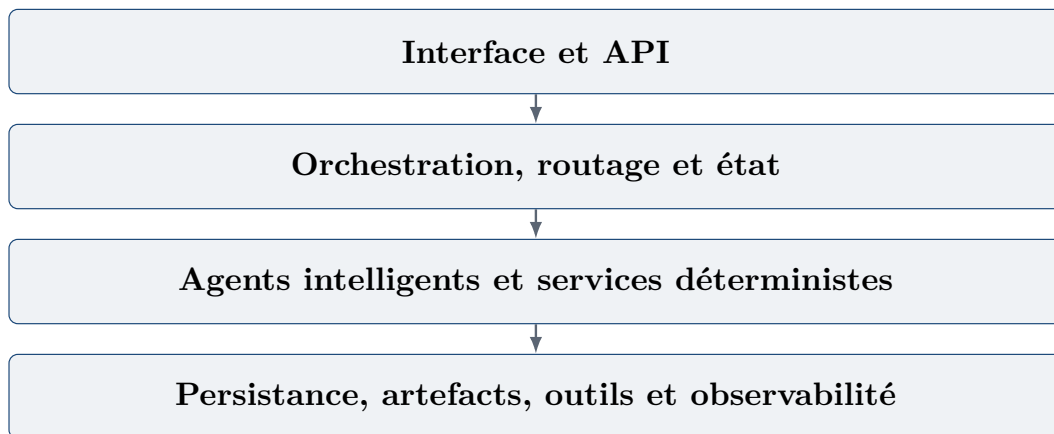


Figure 2.3 – Architecture en couches d'une plateforme agentique

L'humain dans la boucle doit intervenir sur un artefact précis, comprendre les conséquences de son choix et laisser une décision traçable. Une simple confirmation symbolique ne suffit pas.

2.5 Migration logicielle : motivations, types et processus

2.5.1 Définitions et motivations

La mise à niveau concerne une version de composant ; la migration transforme un système source vers une cible ; la modernisation englobe également architecture, sécurité et dette technique ; la refactorisation améliore la structure interne sans changement fonctionnel intentionnel.

Les migrations sont déclenchées par la fin de support, les vulnérabilités, l'incompatibilité avec les plateformes, la dette technique, la maintenabilité et l'accès à de nouvelles capacités.

2.5.2 Types et processus

Les migrations peuvent concerner runtime, langage, framework, dépendances, plateforme, architecture ou données. Dans ce projet, le processus général suit inventaire, définition de cible, planification, transformation, compilation ou construction, tests, réparation et validation.

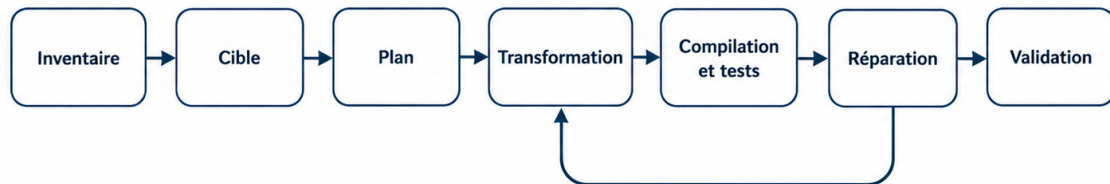


Figure 2.4 – Processus général d'une migration applicative

2.5.3 Spécificités Java/Spring Boot et Angular

Spring Boot associe Java, Spring Framework, dépendances, plugins et propriétés. La progression étudiée couvre Spring Boot 2.1, 2.7, 3.5 et 4.0 avec Java 11, 17 et 21 [30].

Angular associe Angular CLI, Node.js, npm, TypeScript et RxJS. La documentation officielle publie une matrice de compatibilité et recommande de franchir les versions majeures successivement avec l'Update Guide et `ng update` [31, 32]. Le périmètre initial couvre Angular 18 à 21.

2.6 Étude de l'existant et travaux proches

2.6.1 Processus manuel de référence

Avant d'introduire les outils et plateformes étudiés, il est utile de représenter le fonctionnement d'une migration conduite principalement de manière manuelle. Le développeur alterne entre compréhension du projet, consultation de documentation, modification du code et des manifestes, exécution des outils, lecture des erreurs et nouvelle correction. Ce cycle se répète jusqu'à l'obtention d'un état jugé acceptable.

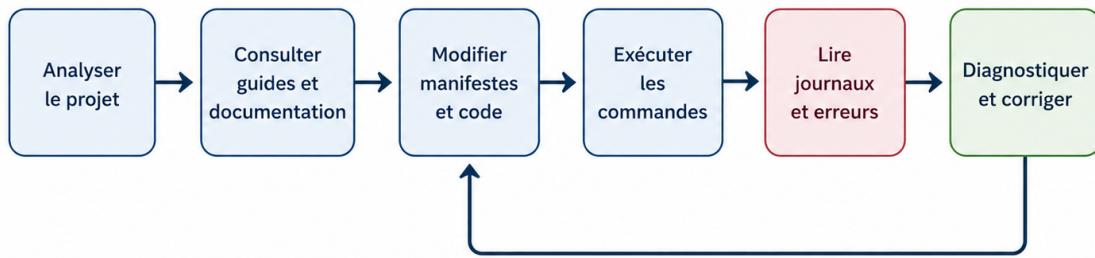


Figure 2.5 – Boucle typique d’une migration conduite manuellement

La difficulté ne réside pas uniquement dans la répétition des opérations. Dans un processus peu instrumenté, l’**état de progression** peut être réparti entre le terminal, les notes du développeur, les branches Git et les tickets ; les **preuves techniques** restent dissociées des décisions qui en découlent ; et les raisons d’une correction ne sont pas toujours rattachées à la commande ou à l’erreur qui l’a motivée.

Cette dispersion a plusieurs conséquences. Elle réduit la **reproductibilité**, car un autre développeur doit reconstruire la séquence exacte des versions, commandes et modifications. Elle limite l’**auditabilité**, puisqu’il devient difficile d’établir pourquoi une décision a été prise et sur quelle preuve. Elle complique également la **reprise** après interruption, le transfert du travail entre intervenants et le **pilotage de plusieurs migrations en parallèle**, faute d’un état consolidé et comparable. Ces limites motivent, dans la Migration Factory, la persistance des états, événements, preuves et décisions ainsi que leur rattachement à une migration identifiée.

2.6.2 Corpus étudié

Le corpus regroupe neuf approches : OpenRewrite, Angular CLI, Dependabot, Amazon Q Developer, GitHub Copilot Modernization, SWE-agent, Repairnator, Auto-Gen et LangGraph. FIPA est étudié séparément comme standard de communication et non comme plateforme d’orchestration.

Repairnator est particulièrement proche de la problématique de réparation : il surveille des échecs d’intégration continue, tente de reproduire les défauts et propose des correctifs aux développeurs. L’étude rapporte cinq correctifs acceptés et fusionnés [41].

SWE-agent évalue l’effet d’une interface agent–ordinateur sur la résolution d’issues. La publication rapporte un taux *pass@1* de 12,5 % sur SWE-bench et de 87,7 % sur HumanEvalFix [39]. Ces valeurs ne sont pas directement comparables à une migration complète, mais elles illustrent l’importance de métriques explicites.

2.6.3 Comparaison synthétique

La comparaison suivante ne cherche pas à établir un classement global entre des solutions de nature différente. Elle examine plutôt cinq dimensions utiles au projet :

le domaine couvert, la présence ou non d'agents, le mode de coordination, les preuves produites et la principale limite au regard d'une migration gouvernée de bout en bout. Cette grille permet de distinguer les capacités directement réutilisables des fonctions qui doivent être combinées ou complétées dans la Migration Factory.

Table 2.3 – Comparaison des travaux et plateformes proches

Approche	Domaine / agents	Orchestration	Métriques ou preuves	Limite principale
OpenRewrite	Migration déterministe, sans LLM	Recettes composables	Diff, compilation et tests du projet	Dépend du catalogue de recettes
Angular CLI	Mise à niveau Angular	Séquence officielle	Installation, construction et tests externes	Périmètre Angular uniquement
Dependabot	Bot de dépendances	Planification périodique	PR créées, CI et taux de fusion	Pas de migration applicative complète
Amazon Q	Agent de transformation Java	Service géré avec itérations	Compilation, tests, diff et durée de transformation	Périmètre et quotas du service
Copilot Modernization	Agents de modernisation	Workflow structuré	Évaluation, plan, compilation, tests et rapport	Service propriétaire
SWE-agent	Agent d'exécution logiciel	Boucle autonome avec outils	12,5 % SWE-bench ; 87,7 % HumanEvalFix	Pas de gouvernance de migration
Repairnator	Bot de réparation	Surveillance CI et outils de réparation	5 correctifs acceptés et fusionnés	Dépend de défauts reproductibles et de tests
AutoGen	Agents conversationnels	Conversations configurables	Benchmarks et études de cas	Persistance et sécurité à concevoir
LangGraph	Orchestration agentique	Graphe d'état et checkpoints	Traces, états et tests applicatifs	Ne fournit pas les agents métier
Migration Factory	Agents spécialisés de migration	Orchestration centrale hybride	Compilation, tests, preuves, décisions et rapports	Prototype local ; évaluation quantitative encore partielle

Le tableau met en évidence des approches complémentaires. Les outils déterministes tels qu'OpenRewrite ou Angular CLI offrent une forte reproductibilité pour des transformations connues, mais ne couvrent pas seuls l'analyse contextuelle, la gouvernance ou la réparation complexe. Les solutions de modernisation assistées par IA ajoutent une capacité d'interprétation et de génération, mais leur périmètre, leur niveau d'autonomie et leurs mécanismes de contrôle varient. Les frameworks d'orchestration comme AutoGen ou LangGraph fournissent quant à eux des mécanismes de coordination, sans définir la sémantique métier d'une migration ni les critères techniques de validation.

La Migration Factory reprend donc plusieurs principes sans chercher à reproduire une solution existante : **exécution déterministe** pour les opérations vérifiables, **agents spécialisés** pour le raisonnement contextualisé, **orchestration persistante** pour l'ordre et les reprises, et **supervision humaine** pour les décisions sensibles. Son positionnement repose ainsi sur l'assemblage gouverné de capacités que le corpus traite souvent séparément.

2.6.4 Carte de positionnement

La carte de positionnement regroupe les solutions étudiées par **famille de contribution** plutôt que par technologie ou éditeur. Elle permet de visualiser les écosystèmes intellectuels et techniques auxquels se rattache la solution : transformation déterministe, assistance intelligente et réparation, orchestration multi-agents, ainsi que standards de communication et d'interaction.

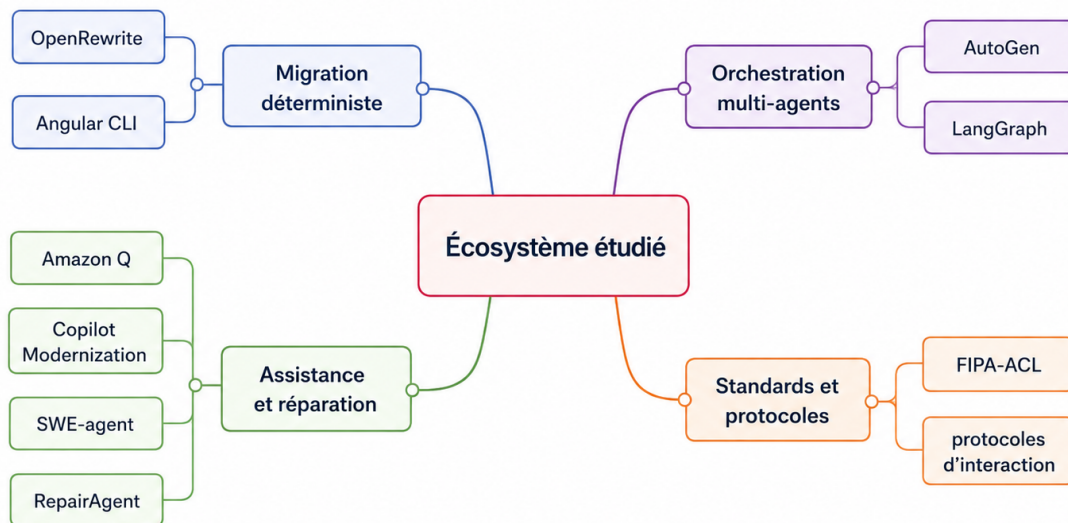


Figure 2.6 – Regroupement des approches, plateformes et standards étudiés

Ces quatre familles ne constituent pas des écosystèmes concurrents au sens strict. La **migration déterministe** apporte des règles répétables et des outils proches du code ; l'**assistance et la réparation intelligentes** apportent une capacité d'adaptation lorsque les règles préétablies sont insuffisantes ; l'**orchestration multi-agents** fournit des structures pour spécialiser les rôles, partager un état et coordonner plusieurs raisonnements ; enfin, les **standards et protocoles** fournissent des principes de communication, de responsabilité et d'interopérabilité.

La réflexion menée dans ce projet consiste précisément à ne pas remplacer un de ces écosystèmes par un autre. Une migration industrielle nécessite à la fois des transformations reproductibles, une adaptation au contexte applicatif, une coordination explicite et des mécanismes de contrôle. La conception retenue cherche donc à faire coopérer ces familles tout en maintenant une frontière nette entre ce qui relève du raisonnement probabiliste et ce qui doit rester déterministe et vérifiable.

2.7 Positionnement de la Migration Factory et choix d'intégration

2.7.1 Synthèse du positionnement

La Migration Factory se situe à l'intersection des transformations déterministes, de l'assistance intelligente et de l'orchestration multi-agents. Elle sépare proposition, révision, décision et exécution ; protège la source dans des espaces isolés ; valide avec les commandes réelles de l'écosystème ; conserve l'état et relie les décisions aux preuves.

Dans le corpus étudié, aucune approche ne réunit simultanément les trajectoires Java et Angular, la spécialisation multi-agents, l'isolation, la réparation gouvernée, les preuves et la supervision humaine. Cette conclusion reste limitée aux solutions analysées et ne prétend pas couvrir l'ensemble de la recherche ou du marché.

2.7.2 Modèles managés et déploiements d'entreprise

L'intégration d'un LLM dans une plateforme d'entreprise peut suivre plusieurs modèles. Une application peut appeler directement un service public à l'aide d'une clé propre au projet, ou consommer un modèle déployé dans une ressource contrôlée par l'organisation. Dans le second cas, la ressource, l'endpoint, les identités et les déploiements sont gouvernés par l'entreprise, tandis que l'application conserve uniquement les références nécessaires à l'appel.

Azure OpenAI et Microsoft Foundry utilisent des déploiements nommés. L'application adresse le nom du déploiement configuré, et non seulement le nom générique du modèle sous-jacent. Cette séparation permet à l'organisation de remplacer un modèle, de modifier sa capacité ou d'attribuer des déploiements différents à plusieurs responsabilités sans modifier la logique fonctionnelle des agents [17].

Approche	Principe	Conséquence architecturale
Appel direct d'un modèle public	L'application contient la configuration d'un fournisseur et d'un modèle donné.	Couplage plus fort entre le code, le fournisseur et le modèle sélectionné.
Modèle managé par l'organisation	Le modèle est déployé dans une ressource appartenant à l'entreprise.	Gouvernance des accès, des déploiements, des quotas et des politiques par l'organisation.
Routage par rôle	Plusieurs déploiements sont associés aux rôles de proposition, de révision ou d'assistance.	Spécialisation des modèles et remplacement indépendant de chaque déploiement.
Configuration externalisée	Endpoint, authentification et noms de déploiement sont fournis au runtime.	Réduction des valeurs codées en dur et meilleure adaptation aux environnements client.

Table 2.4 – Modes d'intégration des modèles dans une plateforme d'entreprise

Cette organisation ne supprime pas les risques. Elle exige une gestion rigoureuse des secrets, des permissions, des journaux et des indisponibilités. Microsoft recommande l'authentification par Microsoft Entra ID pour les connexions sans clé, tout en maintenant la prise en charge des clés API selon le contexte d'exécution [18, 19].

Conclusion

Ce chapitre a distingué agents, services et outils, présenté les fondements des SMA et clarifié l'orchestration centralisée retenue. L'état de l'art inclut désormais des métriques explicites lorsque les publications les fournissent, ainsi que Repairnator pour la réparation automatique. FIPA est correctement positionné comme standard de communication.

Le chapitre suivant traduit ces fondements en une conception détaillée de l'architecture multi-agents, de l'autonomie des agents, des services déterministes et des interactions gouvernées.

Conception de l'architecture multi-agents

Introduction

Le chapitre précédent a montré que la Migration Factory se situe à l'intersection de la migration déterministe, de l'assistance intelligente et de l'orchestration multi-agents. Le présent chapitre franchit l'étape suivante : il transforme ce positionnement conceptuel en **décisions de conception** propres au projet. Il ne s'agit plus seulement d'identifier les capacités utiles, mais de déterminer comment répartir les responsabilités, limiter l'autorité des agents, organiser les échanges, persister l'état et garantir qu'une décision probabiliste ne puisse pas contourner les contrôles techniques.

La conception doit répondre simultanément à plusieurs contraintes. Une migration impose un ordre de stages et de validations ; les outils d'exécution diffèrent entre Java/Spring Boot et Angular ; certains traitements nécessitent du raisonnement contextuel ; les commandes, écritures et changements sensibles doivent rester vérifiables ; enfin, le développeur doit pouvoir interrompre, réviser et reprendre le processus à partir d'un état traçable. Ces contraintes conduisent à séparer les agents intelligents, l'orchestrateur, les services déterministes, la persistance et les points de décision humaine.

La description suit plusieurs points de vue conformément à ISO/IEC/IEEE 42010 : contexte, fonctions, comportement, information, déploiement et gouvernance [2]. Les diagrammes utilisent les principes d'UML 2.5.1 [7]. L'objectif est de montrer les décisions structurantes et leurs justifications, puis de relier ces décisions aux workflows Java et Angular, sans reproduire toutes les classes du code.

3.1 Cadre de conception et choix du modèle SMA

3.1.1 Points de vue architecturaux

Une architecture de cette nature ne peut pas être comprise à partir d'un seul diagramme. La vue fonctionnelle répond à la question « qui fait quoi ? », la vue comportementale montre « dans quel ordre et sous quelles conditions ? », tandis que les vues informationnelle, de déploiement et de gouvernance précisent respectivement les données échangées, l'environnement d'exécution et les limites d'autorité. Le tableau suivant associe chaque préoccupation à la représentation utilisée dans le rapport.

Point de vue	Préoccupation	Modèles utilisés
Contexte	Frontières, acteurs et services externes.	Diagramme de contexte du Chapitre I.
Fonctionnel	Responsabilités des agents et services.	Classification, matrice de responsabilités.
Comportemental	Ordre des tâches, décisions et reprises.	Séquences, activités et machine à états.
Information	États, messages, artefacts et preuves.	Modèle de contrats et mémoire.
Déploiement	Processus locaux, outils et stockage.	Vue de déploiement du Chapitre IV.
Gouvernance	Autorité, approbation et risques.	Matrice d'autonomie et boucle de réparation.

Table 3.1 – Points de vue retenus pour la description de l'architecture

3.1.2 Choix du modèle SMA

Le choix du modèle SMA résulte des contraintes identifiées précédemment plutôt que d'une préférence pour une architecture multi-agents en elle-même. Trois alternatives ont été écartées comme modèle principal : un agent unique concentrant analyse, transformation et exécution ; une chorégraphie entièrement distribuée où chaque composant ferait progresser le workflow localement ; et une automatisation exclusivement déterministe, insuffisante pour certaines analyses contextuelles. Le modèle retenu cherche donc un compromis entre spécialisation du raisonnement, contrôle central de la progression et exécution vérifiable.

La Migration Factory adopte ainsi une architecture **hybride, spécialisée et centralement orchestrée**. Elle est hybride parce qu'elle associe agents fondés sur des LLM, services déterministes, outils externes et supervision humaine. Elle est spécialisée parce que chaque agent possède un objectif limité. Elle est centralement orchestrée parce que la progression dépend d'un état persistant et de transitions contrôlées.

Ce choix répond à quatre contraintes principales :

- l'ordre des étapes de migration ne peut pas dépendre d'une conversation libre ;
- les commandes et écritures doivent rester vérifiables ;
- les décisions sensibles doivent pouvoir interrompre le processus ;
- les résultats intermédiaires doivent être persistés et reliés aux preuves.

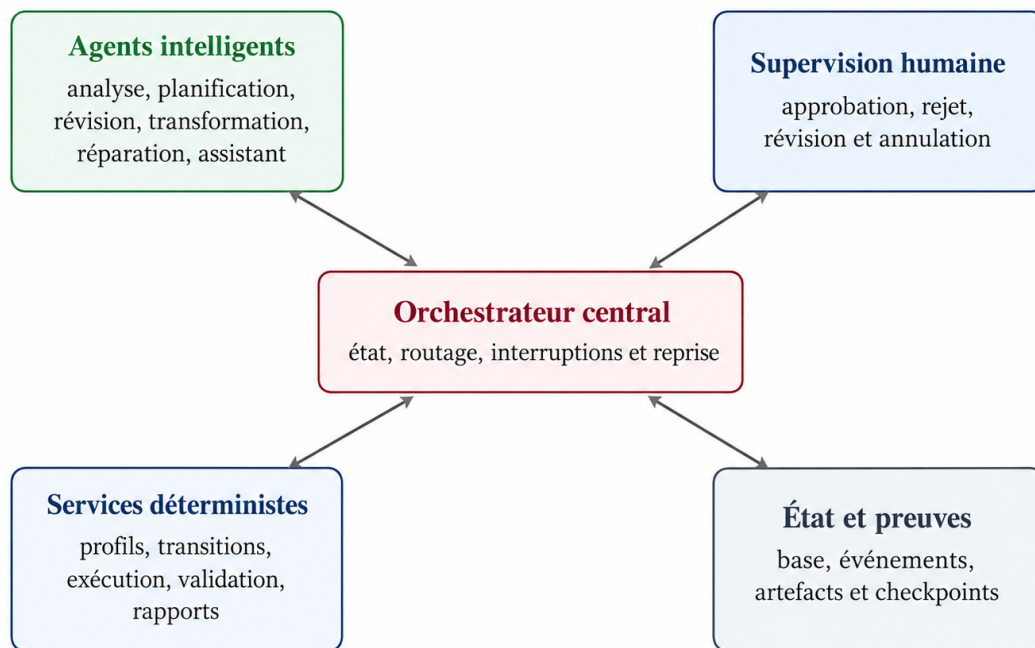


Figure 3.1 – Modèle hybride retenu pour la Migration Factory

Cette architecture commune est ensuite **déclinée dans deux dépôts spécialisés**, l'un pour Java/Spring Boot et l'autre pour Angular. Ce choix ne vise pas à dupliquer arbitrairement la solution ; il répond à la différence structurelle entre les deux écosystèmes.

La première différence concerne les **runtimes et chaînes d'outils**. Java/Spring Boot s'appuie sur des JDK, Maven, `pom.xml`, des plugins Maven et des transformations telles qu'OpenRewrite. Angular dépend de Node.js, npm, Angular CLI, `package.json`, TypeScript et RxJS. Leurs mécanismes de résolution des dépendances, leurs commandes et leurs matrices de compatibilité ne sont donc pas interchangeables.

La deuxième différence concerne le **cycle de validation**. Une étape Java est principalement qualifiée par le profil JDK/Maven, la compilation et les tests Maven. Une étape Angular doit également résoudre une combinaison compatible Angular–Node.js–npm–TypeScript, effectuer une installation reproductible des dépendances, construire l'application et exécuter les tests disponibles. Les causes d'échec, les journaux utiles et les stratégies de réparation ne sont par conséquent pas identiques.

Enfin, conserver deux dépôts limite le **couplage conditionnel** dans l'orchestrateur et les services d'exécution. Une modification du catalogue Node.js ou d'une stratégie npm ne doit pas imposer une régression au moteur Maven, et inversement. Cette séparation réduit le rayon d'impact des changements, facilite les tests propres à chaque écosystème et autorise des rythmes d'évolution indépendants.

Les deux systèmes partagent néanmoins la même **architecture de référence** : spécialisation des agents, orchestration persistante, services déterministes responsables de l'exécution, espaces de travail isolés, preuves liées aux états et décisions humaines sur les opérations sensibles. La généralisation recherchée porte donc sur ces principes et contrats de gouvernance, et non sur l'imposition d'une base de code d'exécution unique.

3.2 Organisation des agents et de l'architecture

Après avoir justifié le modèle SMA, cette partie précise comment ses responsabilités sont réparties entre agents intelligents et services déterministes. Elle présente les composants capables de raisonnement contextualisé, les mécanismes qui conservent l'autorité sur l'exécution, puis la matrice d'autonomie et l'architecture globale qui relie ces deux catégories.

3.2.1 Agents intelligents retenus

Les véritables agents du système sont ceux qui disposent d'une responsabilité de raisonnement ou d'interaction contextualisée :

- agent d'analyse ;
- agent de révision de l'analyse ;
- agent de planification ;
- agent de révision du plan ;
- agent de transformation lorsque la production des changements est agentique ;
- proposeur de réparation ;
- agent de révision de la réparation ;
- assistant conversationnel.

3.2.2 Services déterministes

Les agents ne couvrent volontairement qu'une partie du processus. Les opérations dont le résultat doit être reproductible, contrôlable ou directement vérifiable sont confiées à des services déterministes. Cette séparation empêche notamment qu'une décision probabiliste modifie seule l'état métier, exécute une commande arbitraire ou valide sa propre sortie.

Les composants suivants restent volontairement déterministes :

- résolveur de profils et catalogues de compatibilité ;
- service de transitions et de points de validation ;
- exécuter de commandes ;
- services Maven, npm, compilation, construction et tests ;
- gestion des espaces de travail ;
- stockage des événements et artefacts ;
- projection du cockpit ;

— génération du rapport final.

Cette distinction évite de qualifier d'agent tout composant participant au workflow. Elle préserve également l'autorité des mécanismes vérifiables.

3.2.3 Matrice des rôles et de l'autonomie

La spécialisation des agents n'est utile que si leur périmètre d'action reste explicite. La matrice suivante synthétise, pour chaque rôle, la capacité de raisonnement, l'accès aux outils, les droits d'écriture et les points où une validation indépendante ou humaine demeure obligatoire.

Table 3.2 – Autonomie et limites des agents

Agent	Objectif	Raisonnement	Accès outils	Écriture	Approbation requise
Analyseur	Comprendre le projet et les risques.	Élevé	Lecture bornée	Aucune sur la source	Révision de la sortie
Révision d'analyse	Contrôler couverture et cohérence.	Moyen à élevé	Artefacts uniquement	Aucune	Décision humaine selon le point
Planificateur	Construire trajectoire et unités.	Élevé	Catalogues en lecture	Aucune	Révision et approbation du plan
Révision du plan	Vérifier ordre et pré-requis.	Moyen à élevé	Artefacts uniquement	Aucune	Approbation humaine
Transformateur	Produire des changements candidats.	Élevé	Espace isolé borné	Espace isolé uniquement	Plan approuvé et contrôles
Proposeur de réparation	Générer un diff candidat.	Élevé	Contexte d'échec borné	Aucune application directe	Révision obligatoire
Révision de réparation	Évaluer ou consolider le diff.	Élevé	Preuves et proposition	Aucune application directe	Approbation humaine
Assistant	Expliquer l'état et les actions.	Moyen	Projection en lecture	Aucune	Aucune action sensible

3.2.4 Architecture globale

Les responsabilités décrites précédemment doivent maintenant être replacées dans une vue d'ensemble afin de comprendre leurs interactions. Cette architecture globale montre comment le cockpit, l'API, l'orchestrateur, les agents, les services déterministes et les mécanismes de persistance coopèrent au sein d'une même chaîne d'exécution. Elle met également en évidence le principe d'autorité retenu : les agents produisent des analyses ou des propositions, tandis que l'état, les transitions et l'exécution restent contrôlés par les composants déterministes.

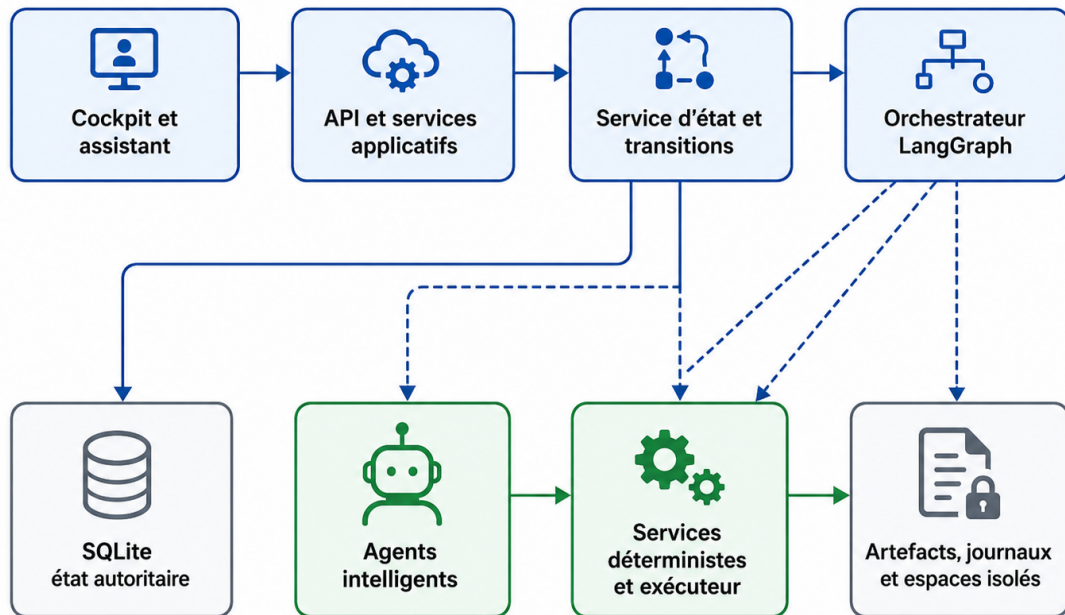


Figure 3.2 – Architecture globale du système multi-agents

La base de données conserve l'état métier autoritaire. LangGraph coordonne les nœuds et les interruptions, mais ne remplace pas les règles métier. Les agents produisent des artefacts ; les services déterministes vérifient, appliquent et exécutent.

3.3 Communication, état et orchestration

Une fois les responsabilités définies, leur coordination devient le point central de l'architecture. Cette partie décrit la manière dont les composants échangent des artefacts structurés, comment l'orchestrateur contrôle la progression et comment l'état persistant permet d'interrompre puis de reprendre une migration sans perdre les décisions déjà validées.

3.3.1 Communication et contrats

Les agents ne maintiennent pas une conversation libre comme source de vérité. Ils échangent indirectement par des artefacts orchestrés. Un contrat logique contient : identifiant de migration, tâche, producteur, consommateur, référence de contexte, schéma attendu, empreintes, statut et erreurs.

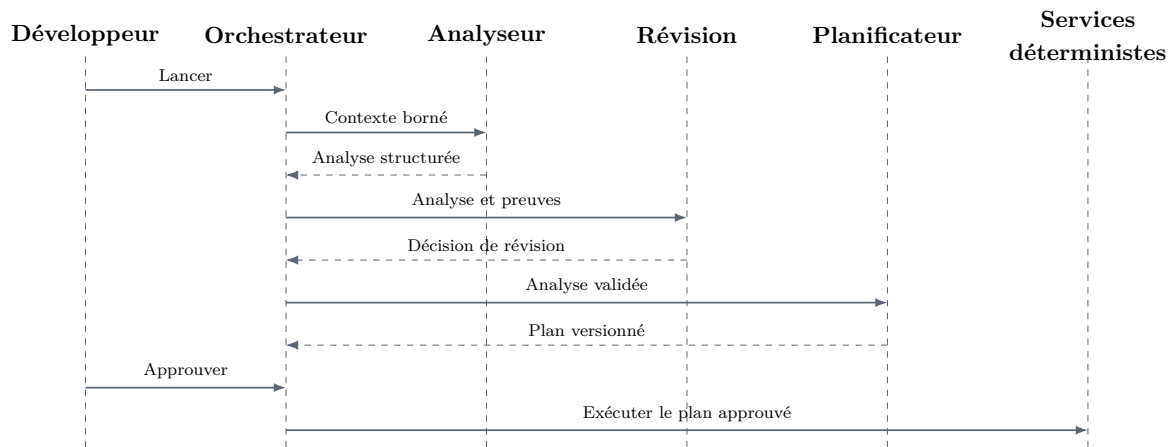


Figure 3.3 – Séquence simplifiée entre agents, orchestrateur et services

3.3.2 Conception de l'orchestrateur

Les contrats définissent ce qui peut être échangé ; l'orchestrateur détermine quand et dans quelles conditions un échange peut faire progresser la migration. Il constitue ainsi le point de coordination entre l'état persistant, les agents, les services déterministes et les décisions humaines.

L'orchestrateur reçoit une intention validée par l'API, charge l'état persistant, sélectionne le nœud autorisé, prépare son contexte, enregistre la sortie, ouvre éventuellement un point de validation et détermine la suite.

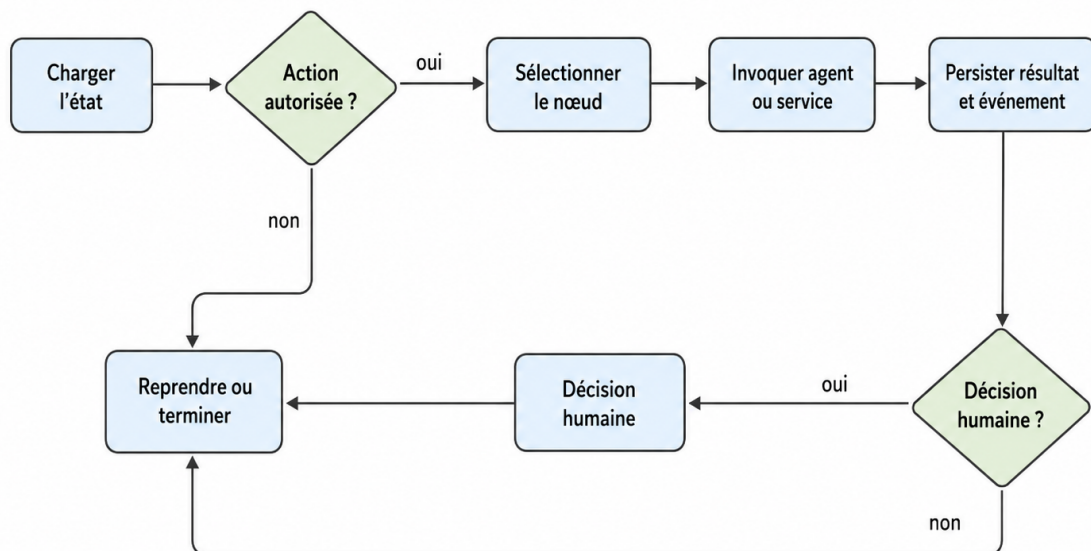


Figure 3.4 – Logique générale de l'orchestrateur

3.3.3 États, mémoire et contexte

Le contrôle du workflow suppose que l'état de la migration ne soit pas reconstruit à partir des seules conversations ou des journaux d'exécution. Il doit être persisté et

accessible aux composants autorisés. La distinction entre état métier, artefacts durables et contexte agentique temporaire soutient ainsi la reprise, la traçabilité et la limitation des informations transmises aux modèles.

Une migration possède un état global et des états locaux pour les stages, commandes, points de validation et réparations. Le frontend présente une projection ; il ne décide pas de l'état.

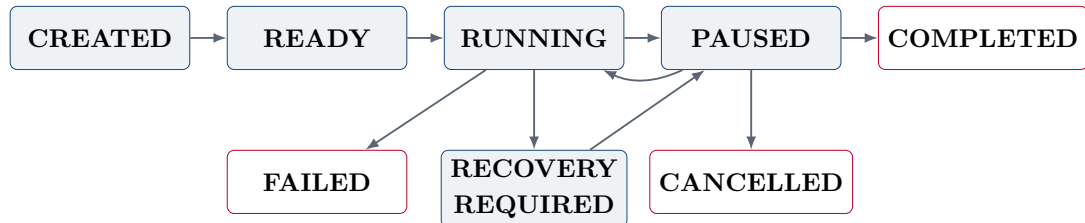


Figure 3.5 – Machine à états simplifiée d'une migration

Trois formes de mémoire sont distinguées : mémoire métier persistante, mémoire documentaire des artefacts et contexte agentique temporaire. Le contexte transmis à un agent est borné, nettoyé et relié à des références persistées. Cette approche soutient la confidentialité et la gestion des risques recommandées par ISO/IEC 23894 et le NIST AI RMF [4, 15, 16].

3.4 Workflows, supervision et reprise

Les mécanismes précédents définissent le cadre commun ; leur utilité apparaît lorsqu'ils sont appliqués aux parcours concrets de migration. Cette partie montre comment ce cadre est décliné pour Java/Spring Boot et Angular, puis comment les décisions humaines, la réparation et la replanification interviennent lorsqu'un stage ne peut pas progresser normalement.

3.4.1 Workflows spécialisés

Java/Spring Boot. Pour chaque transition, le système qualifie JDK et Maven, analyse le projet, révisé l'analyse, planifie, révisé le plan, ouvre une décision humaine, prépare l'espace isolé, transforme, compile et teste. Un échec peut ouvrir la réparation gouvernée avant le passage au stage suivant.

Angular. Le workflow Angular détecte Angular, Node.js, npm, TypeScript et RxJS, résout un profil compatible, prépare un espace de stage propre, analyse manifestes et sources, produit et révisé un plan, applique les transformations, installe les dépendances, construit et teste. L'intégration complète, la réparation et le reporting restent en consolidation.

3.4.2 Supervision et réparation gouvernée

Les points de validation permettent d'approuver, rejeter, demander une révision ou annuler. Une décision est liée à l'état courant et à l'empreinte de l'artefact examiné.

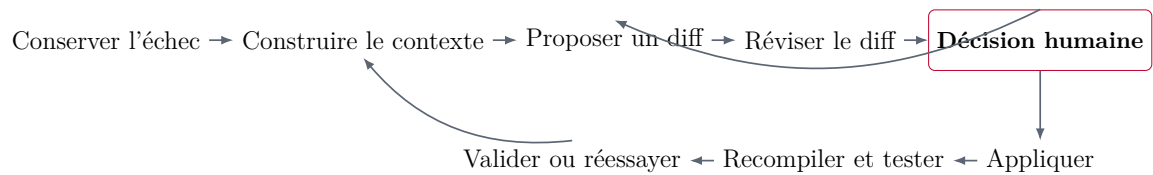


Figure 3.6 – Boucle de réparation gouvernée

Le reviewer de réparation peut consolider la proposition, mais ne peut pas l'appliquer. Le diff final est soumis à des contrôles déterministes, à une approbation humaine et à une revalidation obligatoire. Le nombre de tentatives est limité.

3.4.3 Gestion des erreurs et replanification

Les erreurs sont classées en précondition invalide, erreur transitoire, échec de transformation ou validation, et incohérence d'analyse ou de plan. Selon la catégorie, l'orchestrateur bloque, réessaie, ouvre la réparation, demande une décision ou renvoie vers la planification.

La replanification ne modifie pas silencieusement un plan approuvé : elle crée une nouvelle version qui doit être révisée et décidée.

3.5 Passerelle d'accès aux modèles et routage par rôle

L'accès aux modèles est conçu comme une capacité d'infrastructure distincte des agents. Un agent ne manipule ni la clé d'authentification ni l'endpoint Azure. Il exprime un rôle et fournit un contexte borné ; le backend sélectionne ensuite le déploiement correspondant, construit la requête et valide la réponse avant de la rendre au workflow.

Cette séparation poursuit quatre objectifs : limiter l'exposition des secrets, éviter le couplage d'un agent à un modèle précis, permettre des déploiements différents selon les responsabilités et produire un échec contrôlé lorsqu'une configuration est absente ou indisponible.

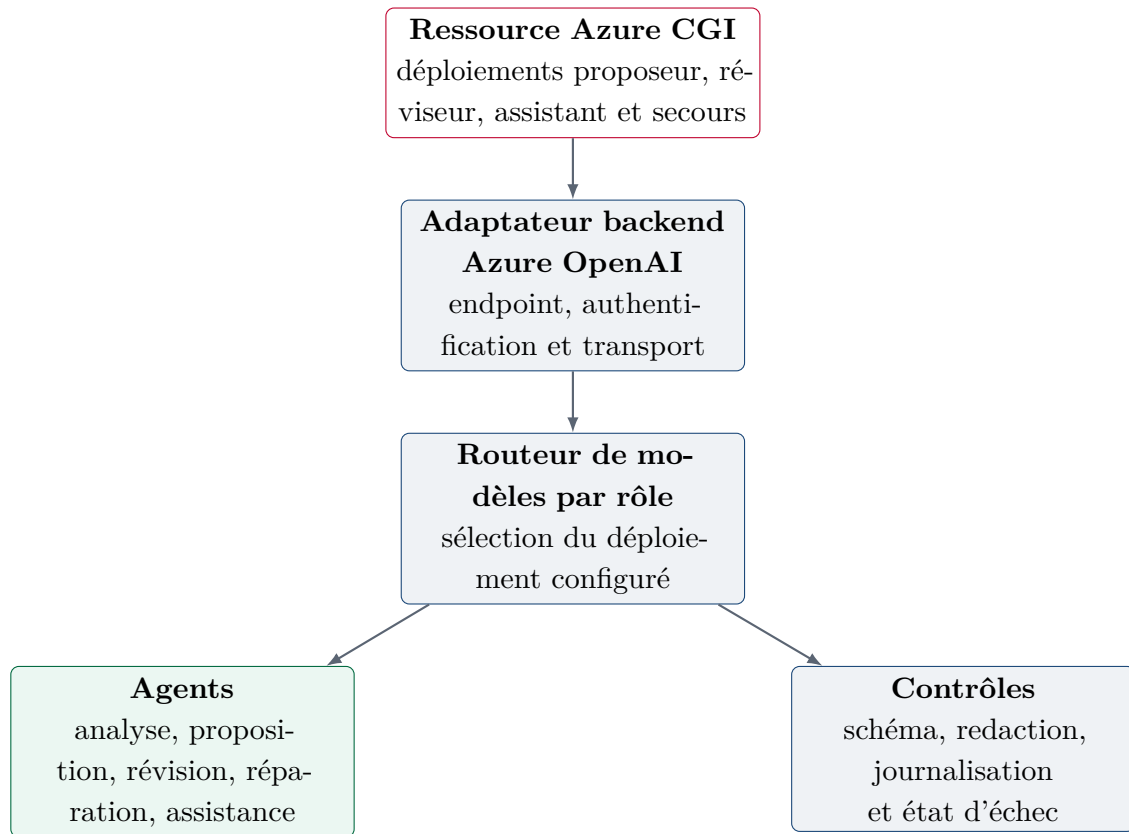


Figure 3.7 – Passerelle Azure OpenAI et routage des déploiements par rôle

Composant	Responsabilité	Information sensible
Ressource Azure de CGI	Héberger et gouverner les modèles déployés.	Oui
Configuration backend	Référencer endpoint, authentification et déploiements.	Oui
Routeur par rôle	Associer une responsabilité agentique au déploiement configuré.	Nom interne uniquement
Agent	Fournir le contexte, le rôle et le schéma de sortie attendu.	Non
Contrôles de sortie	Vérifier le schéma, nettoyer les résumés et persister le statut.	Non après rédaction
Frontend	Afficher un état de disponibilité et des informations non sensibles.	Non

Table 3.3 – Répartition des responsabilités autour de la passerelle Azure

Dans la déclinaison Java observée, le routeur distingue notamment les rôles de proposition, de révision et d'assistance, avec un déploiement de secours optionnel. L'authentification effectivement utilisée par le client courant repose sur une clé API. Microsoft Entra ID constitue une évolution d'industrialisation recommandée, mais ne

doit pas être présenté comme pleinement opérationnel tant que ce mode n'est pas validé dans le runtime du projet [19].

Conclusion

Ce chapitre a défini une architecture SMA où les agents sont réservés aux tâches de raisonnement et d'interaction contextualisée, tandis que les opérations vérifiables restent confiées à des services déterministes. La matrice d'autonomie précise les droits de chaque agent, et les points de vue architecturaux relient les préoccupations aux modèles utilisés.

Le chapitre suivant présente la mise en œuvre concrète : technologies, structure des dépôts, fiches d'agents, orchestrateur, contrats, interface, sécurité, preuves techniques et limitations observées.

Architecture et implémentation du système

Introduction

Le chapitre précédent a défini le modèle multi-agents, les responsabilités et les limites d'autorité. Le présent chapitre décrit la mise en œuvre concrète des deux systèmes de la Migration Factory : environnement technique, évolution du prototype Java, organisation des dépôts, agents, orchestrateur, backend, persistance, interface, sécurité et difficultés rencontrées.

Les deux déclinaisons ne possèdent pas le même niveau de maturité. Java/Spring Boot constitue la référence la plus avancée, avec un processus complet, une réparation gouvernée, un assistant et un reporting. Angular est en développement actif : les agents principaux sont implémentés et la trajectoire progressive 11–21 constitue l'objectif d'extension. La preuve d'exécution versionnée retenue valide actuellement les stages 18 vers 19 et 19 vers 20 ; le stage 20 vers 21 est préparé mais non démarré. Le texte distingue donc les fonctionnalités validées, partielles et encore à consolider.

4.1 Environnement technique

4.1.1 Technologies communes

Avant de distinguer les chaînes propres à Java/Spring Boot et à Angular, il est nécessaire d'identifier le socle technique partagé par les deux déclinaisons. Ce socle regroupe les composants qui assurent le pilotage du workflow, la persistance de l'état, l'exposition des services et l'interaction avec le développeur. Le tableau suivant présente ces technologies communes et précise leur rôle dans l'architecture globale.

Couche	Technologies	Rôle
Interface	React et Next.js	Configuration, cockpit, décisions, assistant et rapports.
Backend	Python et FastAPI	API HTTP, schémas, services applicatifs et projections.
Orchestration	LangGraph	Graphe d'exécution, interruptions, reprises et routage.
Persistance	SQLite et artefacts sur disque	États, événements, décisions, commandes, journaux et rapports.
Intelligence	Modèles et fournisseurs configurés selon le dépôt et le rôle	Analyse, planification, révision, réparation et assistance.
Collaboration	GitHub, Jira, Teams et Confluence	Versionnement, backlog, échanges et documentation.

Table 4.1 – Technologies communes aux deux systèmes

Le rapport ne présente pas Azure OpenAI comme fournisseur commun aux deux dépôts. L'audit Java a confirmé son utilisation dans le runtime observé ; la configuration Angular reste décrite de manière générique tant que son fournisseur courant n'est pas établi par une preuve de code récente.

FastAPI fournit les routes, la validation et l'injection de dépendances [24]. LangGraph coordonne les étapes avec état et checkpoints [25]. SQLite reste adapté au périmètre local ; le mode WAL réduit certains blocages entre lecteurs et écrivain [27].

4.1.2 Environnement Java/Spring Boot

La chaîne Java mobilise JDK 11, 17 et 21, Maven, des profils Spring Boot et OpenRewrite. Les recettes OpenRewrite sont composables et doivent s'abstenir lorsqu'elles ne peuvent pas établir la sûreté d'une transformation [28, 29]. Plusieurs scripts PowerShell facilitent le lancement et le diagnostic sous Windows.

La source Java est fournie par un chemin local. Le système vérifie le projet, les JDK, Maven, le profil de migration et le répertoire de sortie avant la création de l'exécution.

4.1.3 Environnement Angular

La chaîne Angular mobilise Node.js, npm, Angular CLI, TypeScript et RxJS. La matrice officielle Angular relie chaque version à des plages compatibles de Node.js, TypeScript et RxJS [31]. L'installation, la construction et les tests utilisent les cibles du projet [34, 35].

L'entrée prévue est un paquet ZIP contenant l'application source. Le backend crée un snapshot en lecture seule, puis prépare un espace de stage en excluant `node_modules`, `dist`, les caches, les couvertures et les sorties régénérables.

4.2 Évolution technique de la solution Java/Spring Boot

L'architecture actuelle résulte de plusieurs incréments successifs. Cette évolution permet de relier les choix techniques aux limites constatées lors des démonstrations.

4.2.1 Première version : preuve de concept en ligne de commande

La première version opérationnelle était principalement utilisée en ligne de commande. Elle associait des scripts de lancement, des agents d'analyse et de planification, des transformations, l'exécution de Maven, l'analyse des tests et la génération d'artefacts. Cette version a validé le principe d'une migration progressive exécutée dans un espace isolé.

La démonstration du 25 mai 2026 a confirmé la faisabilité technique du workflow. Ses limites étaient surtout liées au pilotage : l'utilisateur devait suivre plusieurs commandes, parcourir les artefacts et reconstruire mentalement l'état global de la migration.

4.2.2 Consolidation du prototype CLI

Entre le 25 mai et le 3 juin, le workflow a été corrigé et stabilisé. Les sorties ont été enrichies, le scénario de démonstration a été mieux maîtrisé et plusieurs contrôles ont été renforcés. La deuxième démonstration, réalisée le 3 juin devant une audience CGI élargie incluant davantage de responsables et directeurs, a présenté cette version consolidée.

Cette étape n'a pas encore introduit la Control Tower complète. Elle a surtout validé la robustesse croissante du prototype et confirmé l'intérêt de construire une interface de pilotage.

4.2.3 Passage à la Control Tower full stack

La troisième étape a transformé la preuve de concept en plateforme. Un backend FastAPI a centralisé les API, les transitions, la persistance et les projections. SQLite a conservé les migrations, stages, commandes, événements et décisions. Un frontend React/Next.js a fourni un cockpit, tandis qu'un chatbot contextualisé a permis d'expliquer l'état courant.

La démonstration du 19 juin 2026 a présenté cette Control Tower full stack : backend, frontend, persistance, pipeline, points de décision, journaux et assistant. Cette version a déplacé le projet d'une logique d'outil technique vers une logique de plateforme de migration.

4.2.4 Consolidation de la V1 gouvernée

Les travaux engagés à partir du 20 juin préparent une V1 prévue pour septembre 2026. Ils portent sur :

- un nouveau frontend améliorant la visualisation du workflow et des preuves ;
- une boucle de réparation séparant le proposeur et l’agent de révision ;
- une approbation humaine liée à l’empreinte exacte du correctif ;
- une application déterministe dans un espace isolé ;
- une nouvelle compilation et l’exécution des tests après chaque correctif ;
- une amélioration des prompts, du contexte, des sorties structurées et de l’exploitation des LLM ;
- un reporting enrichi et une meilleure observation de la consommation des modèles.

Cette quatrième étape est encore en cours à la date de référence du rapport ; elle ne doit donc pas être présentée comme une démonstration déjà réalisée.



Figure 4.1 – Évolution technique de la solution Java/Spring Boot

Dimension	J1 – 25 mai	J2 – 3 juin	J3 – 19 juin	J4 – septembre
Interface	Ligne de commande	CLI améliorée	Frontend web	Nouveau frontend
Backend	Workflow local	Workflow stabilisé	FastAPI et SQLite	Services consolidés
Assistant	Non intégré	Assistance limitée	Chatbot contextualisé	Assistant enrichi
Réparation	Correction manuelle	Diagnostic amélioré	Premiers composants	Boucle gouvernée complète
LLM	Premiers agents	Agents consolidés	Agents intégrés à la plateforme	Contexte et sorties renforcés
Validation	Maven et tests	Scénario consolidé	Pilotage full stack	Revalidation après correctif
Statut	Réalisé	Réalisé	Réalisé	Planifié

Table 4.2 – Comparaison des principaux incréments Java/Spring Boot

4.3 Structure simplifiée des dépôts

Les arbres suivants représentent une vue simplifiée, et non la liste exhaustive de tous les dossiers.

4.3.1 Dépôt Java/Spring Boot

```
migration-factory-java/  
|-- migration_factory/  
|   |-- control_tower/  
|   |   |-- adapters/fastapi/  
|   |   |-- application/  
|   |   |-- domain/  
|   |   '-- infrastructure/sqlite/  
|   |-- orchestrator/  
|   '-- agents/  
|-- web/control-tower/  
|-- tests/control_tower/  
|-- scripts/  
'-- docs/
```

Le domaine et les services applicatifs sont séparés des adaptateurs HTTP et SQLite. Les agents sont isolés des composants d'exécution et ne pilotent pas directement le frontend.

4.3.2 Dépôt Angular

```
angular-migration/  
|-- backend/  
|   |-- app/  
|   |   |-- domain/  
|   |   |-- services/  
|   |   |-- api/  
|   |   |-- infrastructure/  
|   |   '-- orchestration/  
|   '-- tests/  
|-- frontend/  
|-- goals/  
|-- artifacts/  
|-- scripts/  
'-- tests/
```

Les deux dépôts partagent des principes et une terminologie, mais pas leurs services d'exécution. Cette séparation réduit le couplage entre Maven/JDK et npm/Node.js.

4.3.3 Déploiement local

Les deux sous-sections précédentes ont présenté l'organisation interne des dépôts Java/Spring Boot et Angular. La présente vue complète cette description en montrant

comment leurs composants sont réellement instanciés et reliés pendant l'exécution locale : le navigateur accède au frontend, le frontend dialogue avec le backend, et le backend coordonne persistance, espaces isolés, outils d'écosystème et fournisseur LLM. Elle fait ainsi le lien entre la structure du code et l'architecture d'exécution.

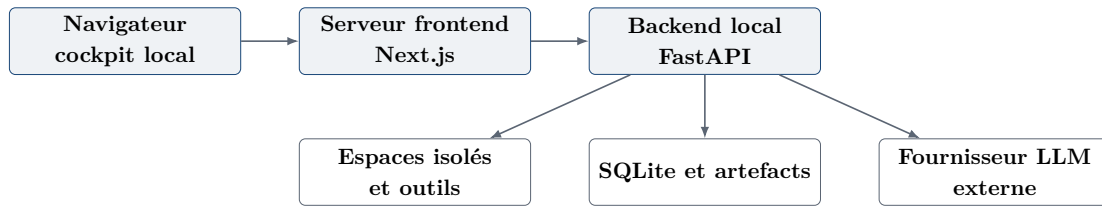


Figure 4.2 – Déploiement local des prototypes

4.4 Implémentation des agents

Chaque véritable agent est présenté selon le même format : objectif, entrées, traitement, sorties, scénario, autorité et limitations.

4.4.1 Agent d'analyse

L'agent d'analyse constitue le point d'entrée du raisonnement agentique. Il transforme les informations brutes du projet en une représentation structurée des versions, dépendances, configurations et risques ; cette sortie sert ensuite de base à l'agent de révision puis au planificateur. Sa qualité conditionne donc directement la pertinence des décisions prises en aval.

Objectif	Construire une représentation exploitable de l'application sans modifier la source.
Entrées	Snapshot, versions, manifestes, structure, configurations et profil.
Traitement	Inventaire déterministe, sélection d'extraits, raisonnement contextualisé et validation de schéma.
Sorties	Rapport d'analyse, risques, références et preuves.
Exemple	Détecter une version Spring Boot ancienne ou une incompatibilité Angular–Node.js.
Autorité	Lecture uniquement ; aucune commande ni écriture sur la source.
Limitations	Dépendances privées et configurations dynamiques peuvent nécessiter une information humaine.

Table 4.3 – Fiche de l'agent d'analyse

4.4.2 Agent de révision de l'analyse

L'agent de révision de l'analyse introduit un premier contrôle indépendant avant toute planification. Son rôle n'est pas de refaire l'analyse, mais d'en vérifier la couverture,

la cohérence et l'appui sur les preuves disponibles. Il se place ainsi entre l'analyseur et le planificateur afin d'éviter qu'une compréhension incomplète du projet ne soit transformée en plan d'exécution.

Objectif	Vérifier couverture, cohérence et exploitabilité du rapport.
Entrées	Analyse, preuves, profil et règles d'acceptation.
Traitement	Comparaison des affirmations avec les preuves et détection des omissions.
Sorties	Acceptation, demande de révision ou rejet motivé.
Exemple	Refuser un rapport Angular qui ne relie pas Angular à Node.js et TypeScript.
Autorité	Ne lance aucune commande et ne modifie pas l'application.
Limitations	Sa qualité dépend des preuves disponibles et du schéma d'analyse.

Table 4.4 – Fiche de l'agent de révision de l'analyse

4.4.3 Agent de planification

L'agent de planification transforme une analyse validée en trajectoire de migration exploitable. Il ordonne les stages, explicite leurs dépendances et associe à chacun des conditions de validation. Il constitue le lien entre la compréhension du projet produite en amont et les opérations de transformation qui seront exécutées après révision et approbation.

Objectif	Décomposer la cible en stages, unités de travail et validations.
Entrées	Analyse validée, cible, catalogue de compatibilité et politiques.
Traitement	Sélection d'une trajectoire, ordonnancement et définition des critères de sortie.
Sorties	Plan versionné, dépendances, risques et points de validation.
Exemple	Construire Spring Boot 2.1 → 2.7 → 3.5 → 4.0.
Autorité	Ne transforme pas le projet ; le plan doit être révisé et approuvé.
Limitations	Dépend de la qualité de l'analyse et des profils disponibles.

Table 4.5 – Fiche de l'agent de planification

4.4.4 Agent de révision du plan

L'agent de révision du plan agit comme second contrôle agentique avant l'exécution. Il vérifie que la trajectoire proposée respecte les prérequis, les compatibilités et l'ordre des transformations attendues. Sa position entre le planificateur et la décision humaine permet de présenter au développeur un plan déjà soumis à une vérification indépendante.

Objectif	Contrôler l'ordre, les dépendances et les validations.
Entrées	Plan, analyse, trajectoire et contraintes.
Traitement	Vérification des prérequis et de la couverture des risques.
Sorties	Acceptation ou demande de nouvelle version.
Exemple	Refuser un saut Angular majeur non autorisé par la trajectoire.
Autorité	Aucune écriture sur le projet et aucune approbation humaine implicite.
Limitations	Ne peut pas prouver la réussite future d'une compilation.

Table 4.6 – Fiche de l'agent de révision du plan

4.4.5 Agent de transformation

L'agent de transformation est le premier agent susceptible de produire des modifications candidates sur l'application migrée. Il intervient uniquement à partir d'un plan validé et dans un espace de travail isolé, sans autorité sur la source d'origine. Il relie ainsi le raisonnement de planification à l'exécution technique, tandis que la compilation et les tests déterministes restent responsables de la validation du résultat.

Objectif	Produire les changements candidats décrits par un plan approuvé.
Entrées	Plan, espace isolé, profil et règles de transformation.
Traitement	Transformations déterministes et génération de changements contextualisés.
Sorties	Fichiers modifiés, diff, métadonnées et preuves.
Exemple	Adapter dépendances, configuration Spring ou code TypeScript.
Autorité	Écriture limitée à l'espace isolé fourni par le backend.
Limitations	Les modifications restent candidates tant que compilation et tests ne réussissent pas.

Table 4.7 – Fiche de l'agent de transformation

4.4.6 Proposeur de réparation

Le proposeur de réparation n'intervient qu'après un échec effectivement observé lors de l'exécution ou de la validation. À partir des journaux, de la commande en échec et d'un contexte borné, il produit un correctif candidat sans pouvoir l'appliquer. Il ouvre ainsi la boucle de réparation tout en restant séparé de l'agent qui révisera la proposition et de l'humain qui décidera de son application.

Objectif	Générer un correctif candidat à partir d'un échec vérifié.
Entrées	Erreurs, commande, extraits, profil et état de l'espace isolé.
Traitement	Diagnostic, sélection des fichiers et production d'un diff structuré.
Sorties	Diff candidat, justification et fichiers concernés.
Exemple	Corriger une incompatibilité de compilation Java.
Autorité	Ne peut ni approuver ni appliquer.
Limitations	Le contexte borné peut omettre une dépendance indirecte.

Table 4.8 – Fiche du proposeur de réparation

4.4.7 Agent de révision de la réparation

L'agent de révision de la réparation constitue le contrôle indépendant de la proposition de correctif. Il examine sa portée, sa cohérence avec l'échec observé et les risques qu'elle introduit ; il peut rejeter la proposition ou produire une version consolidée. Il se situe donc entre le proposeur et la décision humaine, ce qui empêche un même agent de générer puis de valider seul son propre correctif.

Objectif	Évaluer la proposition et produire, si nécessaire, un diff consolidé.
Entrées	Proposition, erreur, politiques et preuves.
Traitement	Contrôle de cohérence, risque, portée et applicabilité.
Sorties	Acceptation, rejet ou diff consolidé.
Exemple	Réduire un correctif trop large à la cause de l'échec.
Autorité	Ne peut pas appliquer ; l'humain décide sur la version exacte.
Limitations	Une sortie correcte en apparence peut encore échouer à la compilation.

Table 4.9 – Fiche de l'agent de révision de la réparation

4.4.8 Assistant de migration

L'assistant de migration joue un rôle transversal d'explication plutôt qu'un rôle d'exécution. Il exploite les états, événements et preuves déjà produits afin d'aider le développeur à comprendre l'avancement, les blocages et les actions disponibles. Contrairement aux agents du pipeline, il ne fait pas progresser la migration et ne dispose d'aucune autorité sur les décisions ou les fichiers.

Objectif	Expliquer l'état, les blocages et les actions possibles.
Entrées	Projection, événements, preuves autorisées et historique borné.
Traitement	Synthèse et réponse contextualisée en lecture seule.
Sorties	Explication, résumé ou recommandation d'action.
Exemple	Expliquer pourquoi une migration attend une approbation.
Autorité	Ne peut ni exécuter, ni approuver, ni modifier des fichiers.
Limitations	Une information absente de l'état ne doit pas être inventée.

Table 4.10 – Fiche de l'assistant de migration

4.5 Services déterministes et orchestrateur

Le résolveur de profils, le service de transitions, l'exécuteur, les services de compilation et de tests, le stockage et le reporting ne sont pas des agents. Ils implémentent des règles vérifiables.

Le frontend transmet une intention à l'API. Le backend valide le schéma, charge l'état et vérifie l'action. L'orchestrateur choisit ensuite le nœud autorisé. Les résultats sont persistés avant toute projection vers le cockpit.

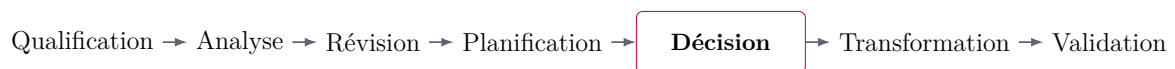


Figure 4.3 – Séquençement principal implémenté par l'orchestrateur

Pour concrétiser ce séquençement, l'orchestrateur s'appuie sur des contrats techniques explicites entre l'API, l'état persistant et les services d'exécution. La représentation suivante illustre le contrat logique d'une décision. Les noms sont simplifiés pour le rapport ; ils ne constituent pas une copie exhaustive d'une route.

```

{
  "run_id": "...",
  "gate_id": "...",
  "decision": "approve | reject | revise",
  "expected_checksum": "sha256:...",
  "idempotency_key": "..."
}
  
```

Un manifeste de commande contient de manière analogue l'exécutable, les arguments validés, le répertoire de travail, l'environnement matérialisé, le délai maximal et les identifiants de corrélation. L'interface ou un LLM ne transmet jamais une commande shell libre.

4.6 Backend, persistance et interface de supervision

Le backend constitue la source d'autorité de la plateforme : il valide les requêtes, persiste les états et rattache les actions aux preuves produites. L'interface ne duplique pas cette logique ; elle expose au développeur une projection de l'état autoritaire afin de rendre le workflow compréhensible et pilotable. Les deux sous-parties suivantes présentent successivement ces responsabilités backend puis leur restitution dans le cockpit.

4.6.1 Backend, persistance et preuves

Les API couvrent configurations, prérequis, migrations, stages, décisions, réparations, événements, artefacts, assistant et rapports. Les schémas typés détectent les sorties incomplètes.

Les entités principales sont Migration, Stage, Commande, Événement, Artefact, Point de validation, Décision, Proposition, Révision et Rapport. Les événements sont séquencés ; les artefacts conservent type, producteur, chemin et empreinte ; les commandes conservent état, durée, code de sortie et journaux.

```
run_<id>/
|-- source_snapshot/
|-- stages/
|   |-- stage_01/workspace/
|   |-- stage_01/analysis/
|   |-- stage_01/planning/
|   '-- stage_01/validation/
|-- repairs/
|-- logs/
|-- evidence/
'-- reports/
```

La source externe n'est pas utilisée comme répertoire d'écriture. Le candidat validé d'un stage devient la base du suivant. Les décisions sensibles sont liées par SHA-256 à l'artefact examiné.

4.6.2 Interface de supervision et captures

Le cockpit présente le résumé, le pipeline, le stage sélectionné, l'analyse, le plan, les points de validation, les commandes, les journaux, les événements, la réparation, l'assistant et les rapports. Le frontend projette l'état fourni par le backend ; il ne maintient pas une vérité parallèle.

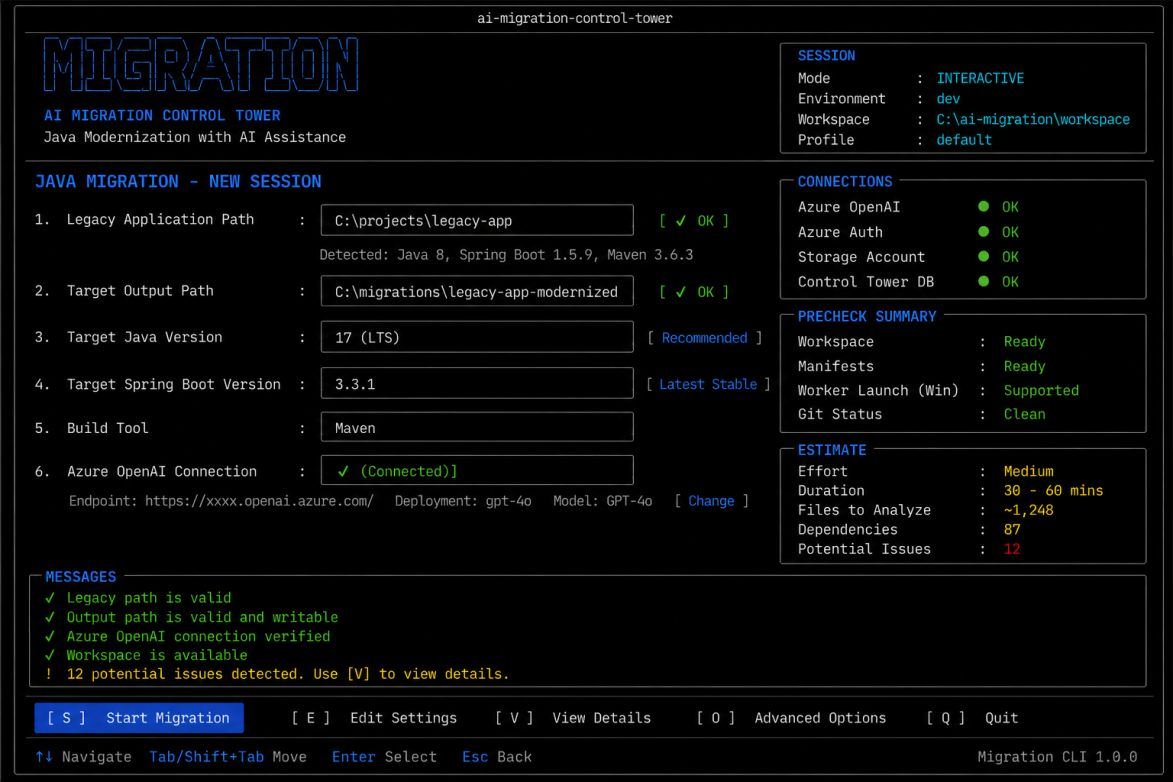


Figure 4.4 – Première version Java/Spring Boot utilisée en ligne de commande

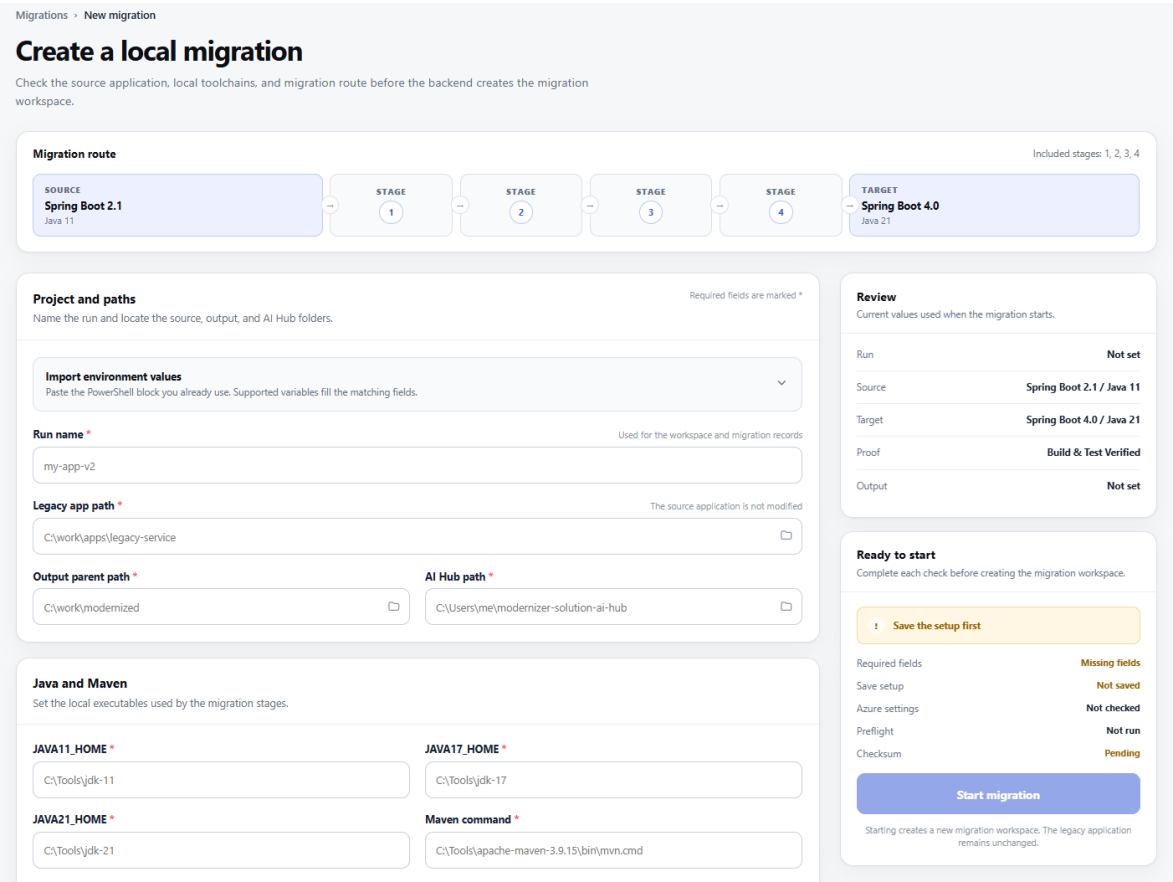


Figure 4.5 – Control Tower Java/Spring Boot

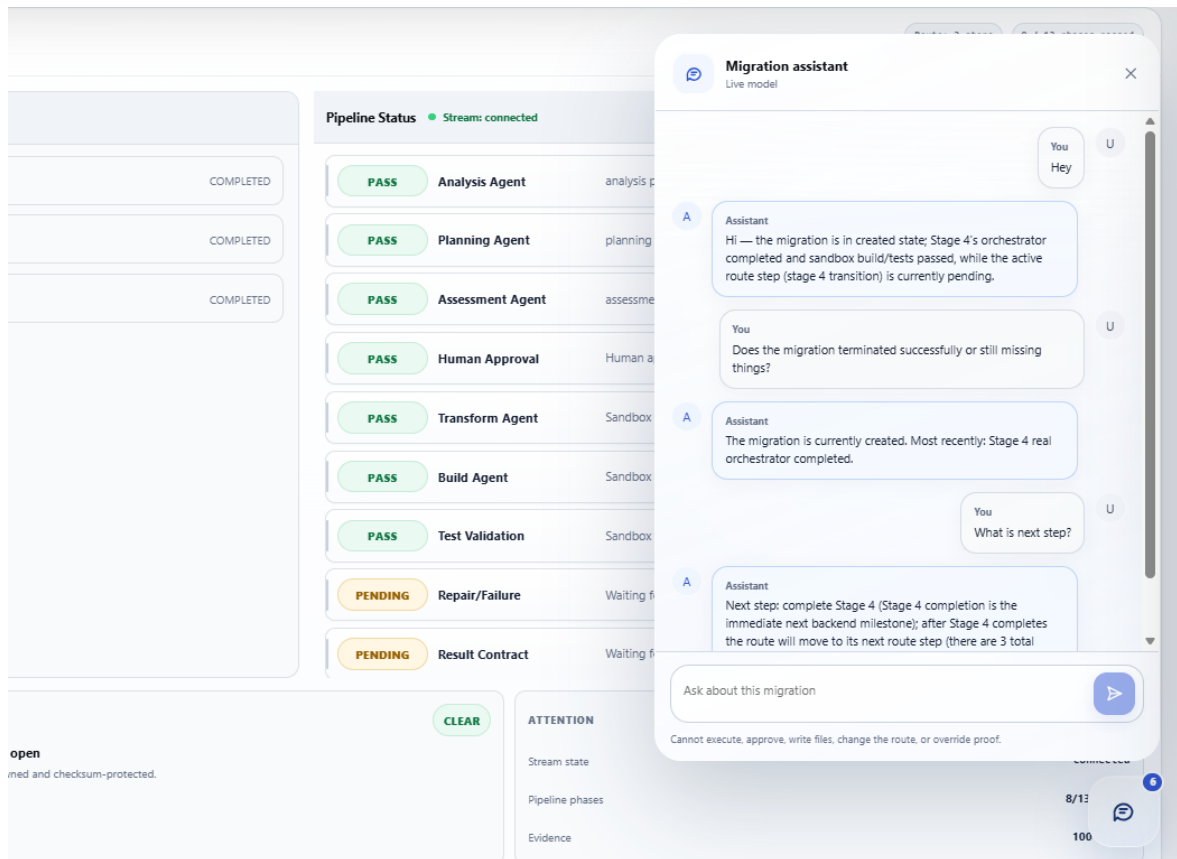


Figure 4.6 – Assistant conversationnel de la Control Tower

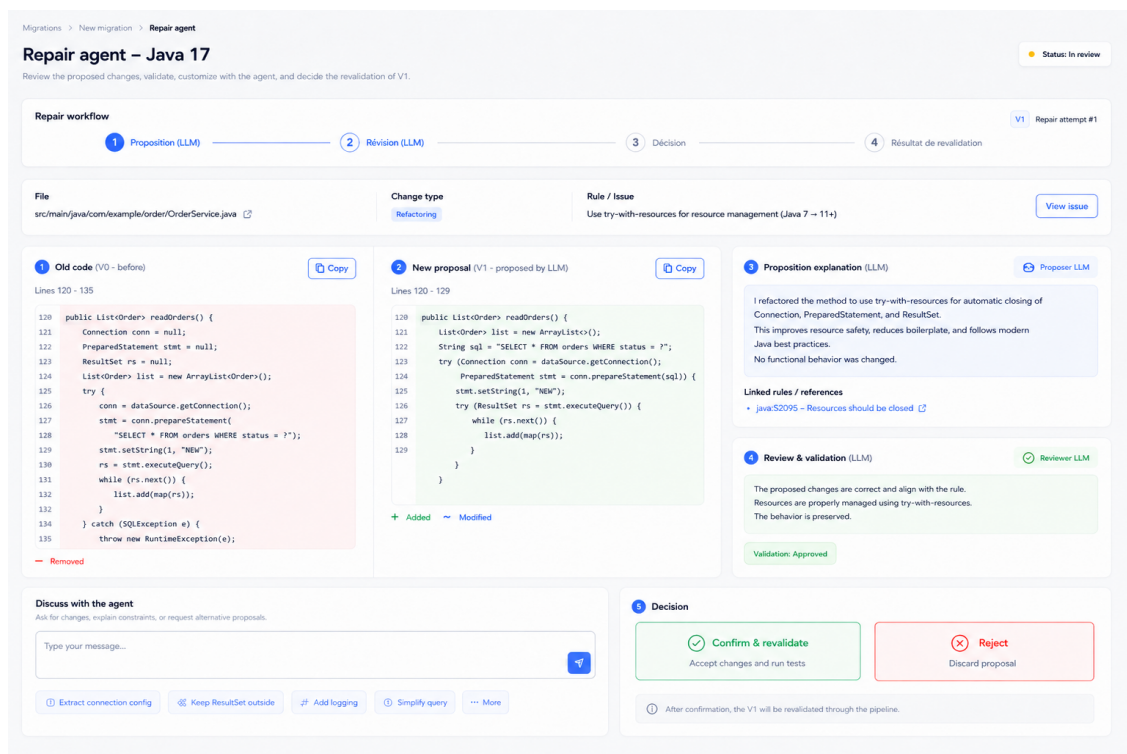


Figure 4.7 – Écran de réparation gouvernée de la V1 Java

MIGRATION PREPARATION

Prepare an authoritative migration

Verify the read-only source boundary, collect readiness evidence, and hand the exact production preflight to the production readiness review.

1 **Project**
Current

2 **Readiness**
Waiting

3 **Source review**
Locked

4 **Create run**
Locked pending production readiness review and approval

STEP 1
Project Configuration revision 0

The source remains read-only. Generated output is run-owned and must stay outside the source project.

Source path

External target-parent path

Target Angular family

Migration mode

Check readiness

STEP 2
Readiness

Path safety and target reservation Waiting
Waiting to verify the read-only source boundary and external target.

Environment capability Waiting
Waiting for path safety before checking the migration environment.

Figure 4.8 – Première démonstration d’avancement Angular

Ces emplacements rendent explicites les preuves visuelles encore à fournir. Aucune capture fictive n’est présentée comme résultat.

4.7 Robustesse, sécurité et validation technique

Après avoir présenté l’exécution fonctionnelle du système, il reste à examiner les conditions qui influencent sa robustesse et son aptitude à être industrialisé. Cette partie regroupe les mécanismes de sécurité déjà présents, les difficultés techniques rencontrées et les preuves de validation disponibles, afin de distinguer clairement les protections implémentées des limites encore ouvertes.

4.7.1 Authentification, sécurité et limites d’exploitation

Les prototypes ne comportent pas d’authentification, de comptes ou de contrôle d’accès par rôle. Le système Java limite l’API au poste local, contrôle l’origine de requêtes

mutatives et utilise un identifiant client statique ; ces mécanismes ne constituent pas une identité fiable.

Une industrialisation devra ajouter authentification forte, rôles, autorisation par action, sessions, révocation et journal d’audit attribuable. Ces évolutions sont cohérentes avec les principes de gestion de la sécurité d’ISO/IEC 27001 [5].

Les contextes LLM sont bornés et nettoyés. La rédaction demeure fondée sur des règles et ne garantit pas la détection de toute valeur sensible intégrée arbitrairement dans le code.

4.7.2 Difficultés techniques et corrections

Difficulté	Cause	Correction
Correctif non applicable	Fins de ligne, hunk ou état obsolète.	Diff structuré, normalisation, empreinte et contrôle d’applicabilité.
Autorité de commande ambiguë	Plusieurs composants pouvaient influencer l’exécution.	Manifeste autoritaire construit par le backend.
Profil Angular rejeté	Catalogue initial incompatible avec Node 22.23.1/npm 10.9.8.	Catalogue versionné et paire explicitement validée.
Base SQLite incohérente	Plusieurs têtes de migration ou mauvaise base active.	Identification de la base autoritaire et réconciliation Alembic.
Intégration frontend	Fusion de nouvelles vues et fonctions existantes.	Intégration progressive et comparaison au baseline.

Table 4.11 – Principales difficultés et corrections

4.7.3 Preuves techniques disponibles

Une campagne d’intégration Java liée au nouveau frontend et à l’assistant a produit 607 tests backend réussis et 4 ignorés. La vérification statique des types, l’analyse de conformité et la construction du frontend ont réussi. Les tests frontend comportaient 12 échecs au baseline et 11 après intégration ; cette comparaison a été utilisée pour distinguer les défauts préexistants d’une régression introduite.

Ces valeurs ne représentent pas un benchmark global de la Migration Factory. Elles constituent une preuve ponctuelle d’intégration. Les scénarios Maven réels restent principalement exécutés manuellement ou en recette utilisateur.

Pour Angular, une migration manuelle d’Angular 18 vers 20 a d’abord été réalisée afin de comprendre les transformations, puis la Factory a été orientée vers une trajectoire 11–21. Les agents d’analyse, de révision, de planification et de transformation sont développés. La preuve d’exécution versionnée retenue établit deux stages scellés, 18

vers 19 puis 19 vers 20 ; le stage 20 vers 21 est préparé mais non démarré. Le processus complet jusqu'à Angular 21 demeure donc en validation.

Aucun des deux prototypes ne dispose actuellement d'une chaîne CI commune dans le dépôt du produit. Il n'existe pas non plus de politique globale unique de lint couvrant toutes les couches. Ces limites sont reprises au Chapitre V.

4.8 Choix d'Azure OpenAI et configuration des modèles du client

Le choix d'Azure OpenAI répond à l'environnement technologique du client et à un objectif de gouvernance. La Migration Factory ne fournit pas un modèle imposé et ne code pas en dur un déploiement unique. Elle permet à CGI de connecter les modèles déjà déployés dans sa propre ressource Azure, puis d'associer un déploiement à chaque responsabilité agentique.

Cette approche peut être décrite comme un principe d'*apport de ses propres déploiements Azure (Bring Your Own Azure Model Deployments)*. L'organisation conserve la maîtrise de la ressource, des modèles, des capacités et des autorisations ; la plateforme consomme uniquement l'endpoint, le mécanisme d'authentification et les noms des déploiements configurés. Les appels Azure utilisent le nom du déploiement déclaré dans la ressource, et non uniquement le nom commercial du modèle [17].

4.8.1 Configuration observée dans la déclinaison Java

L'audit du dépôt Java confirme une configuration externalisée par variables d'environnement. Les agents ne reçoivent pas directement ces valeurs ; elles sont lues par les composants backend responsables du routage et du transport.

Variable	Valeur fournie	Usage
AZURE_OPENAI_ENDPOINT	URL de la ressource Azure	Point d'accès utilisé par le client backend.
AZURE_OPENAI_API_KEY	Clé secrète de la ressource	Authentification effectivement utilisée dans le runtime observé.
AZURE_OPENAI_PROPOSER_DEPLOYME	Nom du déploiement proposeur	Génération des propositions ou correctifs candidats.
AZURE_OPENAI_REVIEWER_DEPLOYME	Nom du déploiement réviseur	Évaluation indépendante et consolidation des sorties.
AZURE_OPENAI_ASSISTANT_DEPLOYM	Nom du déploiement assistant	Réponses conversationnelles contextualisées.
AZURE_OPENAI_FALLBACK_DEPLOYME	Nom optionnel d'un déploiement de secours	Continuité contrôlée lorsque le rôle principal n'est pas disponible et que le secours est activé.

Table 4.12 – Configuration Azure OpenAI externalisée dans le système Java

Le routeur sélectionne le déploiement correspondant au rôle demandé. Les tests du dépôt vérifient notamment l'utilisation du déploiement configuré pour le proposeur ou le réviseur, l'absence de sélection silencieuse lorsqu'une valeur manque et le comportement du secours optionnel.

4.8.2 Chaîne d'appel et contrôle préalable

Après avoir défini les paramètres externalisés et le routage par rôle, il convient de préciser comment une requête traverse effectivement le backend. La chaîne suivante met en évidence les contrôles appliqués avant l'appel au modèle, la sélection du déploiement correspondant au rôle demandé, puis la validation de la réponse avant sa restitution au reste du système. Elle relie ainsi la configuration décrite précédemment aux mécanismes de sûreté et de traçabilité de l'exécution.

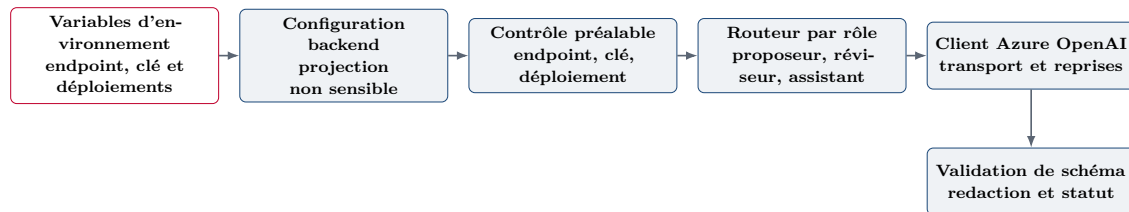


Figure 4.9 – Chaîne d'appel Azure OpenAI dans le backend Java

Le client met en œuvre un appel de contrôle réel pour vérifier l'endpoint, la présence de la clé et le déploiement assistant avant d'annoncer que la configuration est disponible. Les appels de production conservent le rôle, le déploiement utilisé, le statut, les informations de reprise et certains identifiants de diagnostic Azure. Les sorties structurées peuvent être validées par schéma ; une incompatibilité entraîne une fermeture contrôlée plutôt qu'une progression silencieuse.

4.8.3 Protection des informations sensibles

La configuration backend est conçue pour ne pas exposer l'URL complète, la clé, les identifiants de déploiement ou les prompts bruts dans les projections publiques. L'interface reçoit des références de variables, des indicateurs de disponibilité, des rôles et des états de santé. Les mécanismes de redaction nettoient également les résumés d'erreur avant leur affichage.

L'authentification par clé API correspond à l'état réellement observé. Pour une industrialisation, Microsoft Entra ID et les identités managées permettraient une connexion sans clé et éviteraient la gestion directe d'un secret applicatif [18, 19]. Lorsque des secrets restent nécessaires, Azure Key Vault fournit un magasin centralisé et des mécanismes de rotation et de contrôle d'accès [20].

Il convient donc de distinguer :

- **état implémenté** : endpoint, clé API et déploiements nommés externalisés dans le backend Java ;
- **cible d'industrialisation** : Microsoft Entra ID, identité managée, coffre de secrets, rotation et politiques d'accès ;
- **cas Angular** : même orientation fonctionnelle envisagée, mais fournisseur effectif à confirmer par l'audit de la version Angular concernée.

Conclusion

Ce chapitre a décrit l'implémentation en distinguant les agents intelligents des services déterministes. Il a également montré que l'architecture actuelle résulte d'une évolution progressive : prototype CLI, consolidation, passage à la Control Tower full stack et préparation d'une V1 gouvernée.

Les fiches uniformes précisent les entrées, traitements, sorties, exemples, autorités et limites. Les contrats simplifiés, les preuves d'intégration et les difficultés techniques renforcent le caractère concret de la réalisation. Le chapitre suivant formalise les démonstrations et preuves retenues, puis évalue les résultats disponibles, la qualité du système, la comparaison avec le processus manuel et les limites de mesure.

Évaluation, démonstrations et résultats

Introduction

Les chapitres précédents ont présenté les besoins, l'état de l'art, la conception multi-agents et l'implémentation des deux systèmes. Le présent chapitre évalue les résultats disponibles sans confondre preuve observée, validation par démonstration, appréciation qualitative et mesure encore absente.

Le cadre d'évaluation s'inspire d'ISO/IEC 25010, qui propose neuf caractéristiques pour spécifier et évaluer la qualité d'un produit logiciel [3]. Les aspects liés aux modèles génératifs sont également examinés selon une logique de maîtrise des risques cohérente avec le NIST AI RMF et son profil consacré à l'IA générative [15, 16]. L'état présenté est mis à jour au **30 août 2026**. Les résultats techniques sont rapportés selon les preuves effectivement disponibles ; un jalon planifié n'est pas assimilé à un résultat lorsqu'aucun élément de validation correspondant n'est intégré au rapport.

5.1 Objectifs et protocole d'évaluation

Lors de l'évaluation, nous avons cherché à répondre aux questions suivantes :

- le processus protège-t-il la source et limite-t-il les écritures aux espaces isolés ?
- les étapes produisent-elles des preuves exploitables ?
- les agents respectent-ils leurs limites d'autorité ?
- les décisions humaines sont-elles liées aux artefacts examinés ?
- les transformations sont-elles réellement validées par compilation ou construction et tests ?
- l'orchestrateur peut-il suspendre, reprendre et traiter les échecs ?
- le système réduit-il les opérations manuelles répétitives sans masquer les limites restantes ?

Définir le scénario → Exécuter la migration → Collecter états et preuves → Mesurer les critères → Analyser les limites

Figure 5.1 – Protocole d'évaluation retenu

Le protocole idéal prévoit plusieurs applications, plusieurs répétitions et une comparaison avec un processus manuel. La version actuelle du rapport dispose surtout de preuves d'intégration, de scénarios de démonstration et d'observations qualitatives. Les métriques non encore collectées sont explicitement signalées.

5.2 Critères et métriques

Les critères retenus ont été choisis en fonction des objectifs du projet et des risques propres à une migration assistée par des agents. L'intégrité est essentielle pour garantir que la source d'origine reste protégée ; la fiabilité et la traçabilité permettent de vérifier la progression d'un processus composé de plusieurs stages ; l'efficacité mesure la réduction potentielle des opérations répétitives ; la qualité des agents et de la réparation évalue la maîtrise des sorties probabilistes ; enfin, l'utilisabilité et le coût permettent d'apprécier la viabilité de la solution du point de vue du développeur et de son exploitation. Ces dimensions structurent l'analyse des résultats présentée dans les sections suivantes.

Table 5.1 – Critères d'évaluation et disponibilité des mesures

Dimension	Métrique ou preuve	Disponibilité actuelle	Collecte à compléter
Exactitude fonctionnelle	Stage atteint, compilation/construction et tests.	Disponible par exécution.	Agrégation sur plusieurs applications.
Fiabilité	Nombre d'échecs, reprises et états terminaux.	Événements disponibles.	Taux de réussite et répétitions contrôlées.
Intégrité	Empreinte source avant/après et confinement des chemins.	Mécanismes présents.	Rapport automatisé de vérification.
Traçabilité	Proportion d'actions liées à une preuve et un identifiant.	Avancée en Java.	Mesure automatique de couverture.
Efficacité	Durée par stage, attente humaine et appels LLM.	Partiellement journalisée.	Tableau consolidé et comparaison manuelle.
Qualité des agents	Sorties acceptées, révisions, rejets et erreurs de schéma.	Artefacts disponibles.	Taux d'acceptation par agent.
Réparation	Tentatives, correctifs appliqués et validations réussies.	Disponible en Java.	Échantillon plus large et Angular.
Utilisabilité	Actions nécessaires, blocage visible et prochaine action.	Observation du cockpit.	Questionnaire utilisateur structuré.
Coût	Tokens, appels et coût monétaire.	Consommation partielle.	Agrégation complète par migration.

5.3 Validation progressive par les démonstrations CGI

Les démonstrations ont joué un double rôle : vérifier qu'un scénario pouvait être présenté de bout en bout pour le périmètre considéré et recueillir des observations permettant d'orienter l'incrément suivant. Elles constituent des preuves de maturité et de validation par les parties prenantes, mais elles ne remplacent pas une campagne expérimentale reproductible.

Table 5.2 – Jalons de démonstration Java et Angular

Jalon	Date	Public	Périmètre présenté	Statut / conclusion
Java J1	25 mai 2026	Première audience CGI	Première preuve de concept Java/Spring Boot en ligne de commande.	Réalisé ; faisabilité technique validée.
Java J2	3 juin 2026	Audience élargie avec davantage de responsables et directeurs CGI	Version CLI corrigée, enrichie et stabilisée.	Réalisé ; approche consolidée.

Jalon	Date	Public	Périmètre présenté	Statut / conclusion
Java J3	19 juin 2026	Responsables et directeurs CGI	Control Tower full stack : backend, frontend, cockpit, persistance et chatbot.	Réalisé ; passage à une plateforme intégrée.
Angular A1	11 août 2026	Parties prenantes CGI	Première version d'avancement Angular 18 vers 21.	Jalon prévu au planning initial ; non utilisé ici comme preuve technique.
Java J4	Septembre 2026	Public à confirmer	V1 Java : nouveau frontend, réparation gouvernée et agents LLM améliorés.	Planifié.
Angular A2	Septembre 2026	Public à confirmer	Extension progressive de la migration Angular 11 vers 21.	Planifié.

Les démonstrations ont également servi de point de départ à plusieurs évolutions du produit. Les changements apportés après une démonstration ne sont pas tous des demandes explicites des directeurs. Certains résultent de retours recueillis, d'autres d'une décision technique interne prise après l'observation des limites de l'incrément.

Observation ou besoin	Origine	Évolution associée
Suivi difficile d'un workflow uniquement en CLI	Observation de l'usage et retours de démonstration	Création du backend, de la persistance et du cockpit.
Manque de visibilité sur les étapes et blocages	Retour fonctionnel et décision interne	Ajout d'événements, d'états, de projections et de vues du pipeline.
Besoin d'explications contextualisées	Besoin utilisateur observé	Intégration du chatbot puis enrichissement de l'assistant.
Risque d'application directe d'une proposition LLM	Décision d'architecture et de gouvernance	Séparation proposeur/réviseur, empreinte et approbation humaine.
Interface devenue insuffisante face à la richesse du workflow	Décision technique interne	Conception d'un nouveau frontend orienté cockpit.
Périmètre limité à Java/Spring Boot	Objectif de généralisation	Création d'un système Angular indépendant.
Périmètre Angular 18–21 trop restreint pour certains cas réels	Évolution de l'objectif fonctionnel	Extension planifiée vers Angular 11–21.

Table 5.3 – Lien entre observations, décisions et évolutions du produit

Au-delà de ces évolutions, il convient de préciser la portée de ce type de validation. Une démonstration prouve qu'un scénario a été suffisamment intégré pour être présenté

et examiné par les parties prenantes. Elle peut valider une orientation, révéler un problème d'utilisabilité et déclencher une adaptation du backlog. Elle ne démontre pas automatiquement un taux de réussite général, une performance moyenne, une sécurité de production ou une réduction mesurée du temps de migration. Ces conclusions nécessitent des expériences répétées, des configurations publiées et des métriques consolidées.

5.4 Résultats Java/Spring Boot

La déclinaison Java constitue le système le plus complet et fournit le premier niveau de résultats observés. Elle couvre la qualification, l'analyse, la planification, les points de validation, les espaces isolés, les transformations, Maven, les tests, la réparation gouvernée, le cockpit, l'assistant et le rapport final.

Le parcours progressif réduit la distance entre deux validations. Une migration n'est pas considérée comme réussie parce qu'un agent l'affirme : le statut dépend des commandes exécutées, de leur code de sortie et des tests disponibles.

Cette couverture fonctionnelle est complétée par des preuves d'intégration issues d'une campagne ciblée. Une campagne d'intégration liée au nouveau frontend et à l'assistant a produit les résultats suivants :

Vérification	Résultat observé	Interprétation
Suite backend	607 réussites, 4 tests ignorés	Preuve ponctuelle de non-régression backend sur cette intégration.
Vérification statique frontend	Réussie	Contrats TypeScript cohérents pour le périmètre modifié.
Analyse de conformité frontend	Réussie	Aucune erreur bloquante dans la commande exécutée.
Construction frontend	Réussie	Application compilable pour la campagne considérée.
Tests frontend	12 échecs au baseline, 11 après intégration	Comparaison utilisée pour séparer défauts préexistants et nouvelle régression.

Table 5.4 – Résultats d'une campagne d'intégration Java

Ces chiffres ne constituent pas un benchmark général. Ils concernent une campagne précise et doivent être accompagnés du commit, de la configuration et des commandes lors de la version finale du rapport.

Un autre résultat structurant concerne la boucle de réparation gouvernée. La réparation Java met en œuvre la proposition, la révision, la décision humaine, l'application déterministe et la revalidation. Les principaux résultats qualitatifs sont :

- les propositions sont représentées par des diffs examinables ;

- les empreintes empêchent l’approbation d’une version obsolète ;
- les correctifs sont appliqués uniquement dans l’espace isolé ;
- la compilation et les tests sont relancés après chaque application ;
- le nombre de tentatives est limité et l’épuisement est persisté.

Le taux de réussite des réparations, la durée moyenne et le nombre moyen de tentatives ne sont pas encore consolidés sur un échantillon suffisant.

5.5 Résultats Angular et extension

5.5.1 État actuel et résultats observés

Le développement Angular a commencé à la mi-juillet 2026 par une phase d’étude destinée à comprendre les commandes de mise à niveau, les contraintes de compatibilité et les validations attendues. Le premier périmètre portait sur Angular 18 vers 21 ; les travaux ont ensuite été étendus à une trajectoire plus large allant d’Angular 11 vers Angular 21, toujours par transitions majeures successives.

La preuve d’exécution versionnée actuellement disponible dans le dépôt Angular établit deux transitions complètes et scellées : Angular 18 vers 19 puis Angular 19 vers 20. Le stage Angular 20 vers 21 a été préparé, mais il a été volontairement arrêté avant l’approbation de démarrage ; aucune commande de migration 20 vers 21 n’est donc présentée comme exécutée. Cette preuve confirme le fonctionnement du mécanisme stage par stage sur le périmètre exercé, sans démontrer à elle seule la couverture complète d’Angular 11 vers Angular 21.

Les essais ont également mis en évidence des conflits de dépendances réels pendant la transition Angular 19 vers 20. La résolution a nécessité la conservation des journaux d’échec, l’ajustement contrôlé du graphe de dépendances, la régénération de l’installation avec `npm ci`, puis une nouvelle vérification des versions, de la construction et des tests avant le scellement du stage. Ce cas illustre l’intérêt de conserver une preuve complète de chaque étape plutôt que de considérer le seul code de sortie d’une commande comme une validation suffisante.

À ce stade, les limites suivantes restent retenues dans l’évaluation :

- le parcours complet Angular 11 vers 21 n’est pas encore démontré de bout en bout ;
- la preuve d’exécution actuellement intégrée valide Angular 18 vers 19 et 19 vers 20, tandis que 20 vers 21 reste préparé mais non démarré ;
- la réparation gouvernée, le reporting et l’assistant Angular doivent encore être consolidés sur l’ensemble du parcours ;
- aucune série quantitative comparable à la campagne Java n’est encore disponible ;
- les démonstrations prévues dans le planning ne sont utilisées comme preuves de résultat que lorsqu’elles sont documentées par des éléments vérifiables.

5.5.2 Extension visée d'Angular 11 vers Angular 21

L'objectif reste d'achever la trajectoire progressive jusqu'à Angular 21. Il ne s'agit pas d'effectuer un saut direct depuis Angular 11 : chaque version majeure constitue un stage distinct, associé à un profil de compatibilité, un espace de travail isolé, une installation propre, une construction et l'exécution des tests disponibles. Cette stratégie suit la recommandation officielle d'Angular consistant à franchir successivement les versions majeures [32, 33].

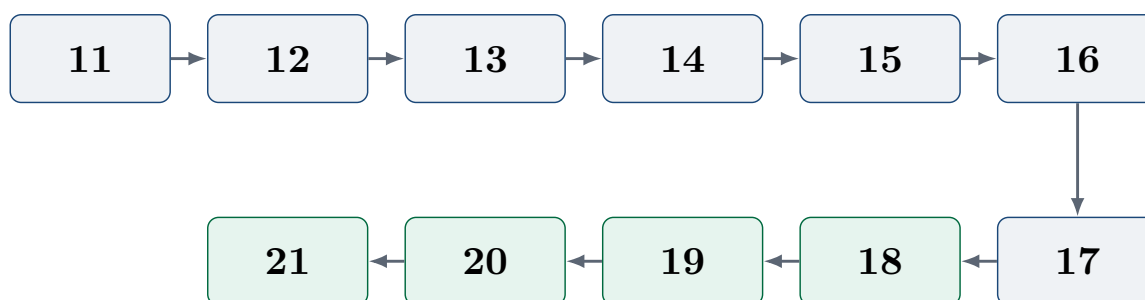


Figure 5.2 – Trajectoire progressive visée pour Angular 11 vers Angular 21

La figure représente la trajectoire cible complète. Les transitions déjà scellées constituent des résultats acquis ; les étapes restantes demeurent des objectifs de validation. Cette distinction permet de conserver une feuille de route unique sans présenter comme réalisées les versions qui ne disposent pas encore d'une preuve de stage scellé.

5.6 Analyse qualitative par caractéristique de qualité

Cette analyse reprend les critères définis en Section 5.2 et les confronte aux preuves présentées dans les Sections 5.3 à 5.5. Les démonstrations CGI apportent une validation progressive par les parties prenantes, tandis que les campagnes d'intégration, les états persistés et les résultats de stages fournissent des éléments techniques plus directement vérifiables. L'objectif n'est donc pas d'attribuer une note globale à la Migration Factory, mais d'identifier, pour chaque caractéristique jugée importante, ce qui est effectivement observé, l'impact de ces mécanismes sur le processus de migration et les limites qui empêchent encore une conclusion quantitative générale.

Caractéristique	Apport observé	Limite actuelle
Adéquation fonctionnelle	Processus couvrant analyse, planification, transformation et validation.	Couverture Angular encore partielle.
Fiabilité	États persistés, préconditions, reprises et preuves.	Certaines opérations concurrentes et reprises restent partielles.
Efficacité	Automatisation d'opérations répétitives et assistance au diagnostic.	Durées et coûts non consolidés.
Compatibilité	Profils de versions explicites et trajectoires par stages.	Installation automatique des runtimes absente.
Utilisabilité	Cockpit, événements, blocages et actions disponibles.	Validation utilisateur structurée à réaliser.
Sécurité	Source protégée, chemins bornés et autorité déterministe.	Pas d'authentification ni de rôles.
Maintenabilité	Agents et services séparés, deux dépôts spécialisés.	Duplication de principes à maintenir.
Portabilité	Profils documentés et exécution locale.	Orientation Windows et scripts PowerShell importants.

Table 5.5 – Analyse qualitative selon des caractéristiques d'ISO/IEC 25010

Le tableau met d'abord en évidence que l'**adéquation fonctionnelle** ne dépend pas uniquement du nombre de fonctionnalités présentes. Dans le contexte d'une migration, une capacité n'est réellement utile que si elle s'insère dans une chaîne contrôlée allant de l'analyse jusqu'à la validation. La déclinaison Java couvre aujourd'hui ce parcours de manière plus complète : analyse, planification, transformation, exécution Maven, tests, décisions humaines, réparation et reporting sont reliés par un état persistant. Angular reprend les mêmes principes, mais la preuve disponible montre encore une progression incomplète vers la cible Angular 21. Cette différence de maturité est importante pour l'interprétation des résultats : elle confirme la transférabilité de l'architecture, sans permettre d'affirmer que les deux déclinaisons disposent déjà du même niveau de couverture.

La **fiabilité** est principalement soutenue par les préconditions, les transitions explicites et la persistance de l'état. Dans un processus manuel, un développeur peut relancer une commande après une correction sans conserver formellement la cause de l'échec, les paramètres exacts ou l'état intermédiaire. La Factory cherche au contraire à rattacher chaque progression à un stage et à des preuves. Le cas Angular illustre directement l'intérêt de cette distinction : une commande de migration peut se terminer sans erreur alors que le stage n'est pas encore scellé. Le système évite ainsi d'assimiler un succès local à une validation globale. Cette propriété réduit le risque de poursuivre sur une base non qualifiée, mais elle dépend encore de la couverture effective des mécanismes

de reprise et de la capacité à reproduire les mêmes conditions d'exécution sur plusieurs environnements.

L'**intégrité**, la **gouvernance** et la **traçabilité** constituent trois dimensions fortement liées. La source externe est protégée et les changements sont produits dans des espaces de travail isolés ; les décisions sensibles sont associées à des artefacts identifiés ; les agents ne disposent pas d'une autorité générale pour exécuter ou approuver leurs propres propositions. Cette séparation réduit le rayon d'impact d'une sortie LLM incorrecte et facilite l'examen humain. Elle apporte également une base d'audit : lorsqu'un plan, une réparation ou une transition est refusé, l'objectif est de pouvoir relier la décision à l'état et aux preuves qui existaient au moment du choix. L'impact est surtout qualitatif à ce stade, car la couverture de traçabilité n'est pas encore exprimée sous la forme d'un indicateur consolidé. Une évolution pertinente consistera à mesurer automatiquement la proportion d'actions critiques disposant d'une preuve, d'un producteur, d'un horodatage et d'une empreinte exploitable.

L'**efficacité** doit être interprétée avec davantage de prudence. La Factory automatise des opérations répétitives telles que la qualification des profils, l'enchaînement des stages, la collecte des journaux et la préparation du contexte de diagnostic. Elle réduit également le besoin de reconstruire manuellement l'état d'une migration à partir du terminal et de plusieurs fichiers. Ces mécanismes indiquent un potentiel de réduction de l'effort humain, mais le rapport ne dispose pas encore d'une comparaison chronométrée sur un corpus identique traité manuellement et avec la Factory. Il serait donc incorrect de convertir ce potentiel en pourcentage de gain de temps. La conclusion défendable est plus limitée : certaines tâches sont désormais prises en charge de manière systématique et traçable, tandis que la mesure du gain global reste à établir expérimentalement.

La **supervision du développeur** joue un rôle complémentaire. L'objectif du projet n'est pas de supprimer l'intervention humaine, mais de déplacer cette intervention vers les décisions qui nécessitent réellement une appréciation. Le cockpit, les points de validation et l'assistant rendent visibles l'état courant, les blocages et les actions possibles. La valeur de cette approche dépend cependant de la qualité de la projection fournie à l'utilisateur. Une interface riche mais incapable de distinguer clairement un stage validé d'un stage seulement exécuté augmenterait au contraire le risque de décision. C'est pourquoi l'utilisabilité doit être évaluée non seulement en termes d'ergonomie, mais aussi en termes de compréhension de l'état, de lisibilité des preuves et de clarté de la prochaine action autorisée.

La **maintenabilité** et la **portabilité** sont directement influencées par le choix de séparer les principes génériques des mécanismes propres aux écosystèmes. Java/Spring Boot et Angular utilisent des dépôts et des chaînes d'outils distincts, mais partagent les mêmes notions d'analyse, de plan, de stage, de validation, d'espace isolé et de gouvernance. Cette séparation limite le couplage entre Maven/JDK et npm/Node.js et permet de faire évoluer un catalogue de compatibilité sans modifier l'autre moteur. En contrepartie, elle crée un besoin de cohérence entre deux implémentations qui évoluent à des rythmes différents. La généralisation de la Factory devra donc porter sur les contrats, les règles de gouvernance et les modèles de preuve avant de chercher à

mutualiser artificiellement les outils d'exécution.

Enfin, l'analyse met en évidence une **asymétrie de maturité** qui doit être conservée dans la présentation des résultats. Java/Spring Boot sert de référence pour les mécanismes les plus avancés, notamment la réparation gouvernée et le reporting. Angular sert davantage à éprouver la capacité d'adaptation du modèle et les difficultés liées aux profils de runtime, aux versions successives et aux validations par stage. Cette asymétrie n'est pas un défaut méthodologique tant qu'elle est explicitée : elle permet au contraire de distinguer ce qui a déjà été éprouvé dans un premier écosystème de ce qui est encore en cours de validation dans le second.

L'impact principal de ces résultats concerne donc l'**industrialisation**. La Factory dispose d'une base structurée pour contrôler l'exécution, isoler les modifications et conserver des preuves, mais plusieurs éléments doivent encore être renforcés avant un usage à plus grande échelle : instrumentation systématique des durées et coûts, authentification et autorisation, validation sur un corpus plus large, exécution répétée dans des environnements contrôlés et consolidation de la portabilité. L'analyse qualitative permet de prioriser ces travaux sans présenter comme acquises des propriétés qui n'ont pas encore fait l'objet d'une mesure suffisante.

5.6.1 Évaluation de l'intégration Azure OpenAI

L'intégration Azure doit être évaluée séparément de la qualité fonctionnelle des agents. Un modèle peut produire une sortie correcte alors que la configuration du fournisseur reste fragile, et inversement. Les critères portent donc sur la configurabilité, la disponibilité, la confidentialité, la traçabilité et la résilience.

Dimension	Preuve attendue	État observé en Java	Validation restante
Configurabilité	Changer un déploiement sans modification du code.	Déploiements par rôle lus depuis l'environnement et couverts par des tests de routage.	Tester plusieurs ressources et configurations client.
Disponibilité	Détecter endpoint, authentification ou déploiement manquant.	Contrôle préalable réel et états d'échec explicites.	Ajouter supervision continue et seuils d'alerte.
Confidentialité	Absence de secrets dans l'interface, les DTO et les résumés publics.	Projection limitée aux références, indicateurs et états ; fonctions de redaction présentes.	Audit de sécurité dédié et tests sur des secrets non standards.
Traçabilité	Conserver rôle, statut, déploiement utilisé et identifiant de diagnostic.	Métadonnées d'invocation et identifiants Azure prévus par le client.	Consolider la restitution dans le rapport final.
Résilience	Comportement contrôlé en cas d'erreur, de délai ou de limite de débit.	Gestion des délais, erreurs HTTP, reprises limitées et secours optionnel.	Mesurer le comportement sous charge et sur des indisponibilités prolongées.
Portabilité d'environnement	Connecter une autre ressource Azure sans refonte.	Structure de configuration compatible avec ce besoin.	Validation multi-tenant, multi-ressource et politiques réseau.
Authentification d'entreprise	Utiliser une identité sans clé et des permissions minimales.	Non validé dans le runtime observé ; clé API utilisée.	Intégrer Microsoft Entra ID, identité managée et coffre de secrets.

Table 5.6 – Critères d'évaluation de l'intégration Azure OpenAI

La structure actuelle permet de connecter une ressource Azure appartenant au client et d'attribuer des déploiements distincts aux rôles. Elle ne prouve cependant pas encore la portabilité sur plusieurs tenants, régions, politiques réseau ou mécanismes d'identité. Ces validations relèvent de l'industrialisation et doivent être réalisées avant un déploiement multi-client.

La déclinaison Angular ne doit pas être incluse dans cette évaluation Azure tant que son fournisseur LLM effectif et ses mécanismes d'authentification n'ont pas été confirmés par le code et les tests correspondants.

5.7 Comparaison avec le processus manuel

La comparaison avec le processus manuel vise à évaluer la valeur opérationnelle de la Factory à partir d'un même enchaînement de tâches : préparer l'environnement, déterminer la trajectoire, exécuter les transformations, diagnostiquer les échecs, corriger puis revalider. L'enjeu n'est pas d'opposer une automatisation complète à un travail humain, puisque le développeur conserve un rôle de décision dans la solution proposée. Il s'agit plutôt d'identifier quelles opérations restent entièrement manuelles dans une migration classique, lesquelles sont structurées ou automatisées par la Factory, et comment cette transformation modifie la traçabilité, la reproductibilité et le risque d'erreur.

Dimension	Processus manuel	Migration Factory
Trajectoire	Construite au cas par cas.	Profils et stages explicites.
Commandes	Exécutées directement par le développeur.	Manifestes construits par le backend.
Isolation	Dépend des pratiques individuelles.	Espaces dédiés et source externe protégée.
Diagnostic	Lecture et recherche manuelles.	Contexte structuré et assistance agentique.
Réparation	Correction directe puis nouvel essai.	Proposition, révision, décision, application et revalidation.
Traçabilité	Dispersée entre terminal, Git et tickets.	États, événements, preuves et décisions reliés.
Mesure	Souvent limitée à la réussite finale.	Possibilité de mesurer stages, commandes, appels et tentatives.

Table 5.7 – Comparaison qualitative avec le processus manuel

Le tableau montre que la différence principale ne réside pas uniquement dans le nombre de commandes exécutées automatiquement. Elle concerne surtout la manière dont ces commandes sont replacées dans un processus explicite. Dans une migration manuelle, la trajectoire peut être reconstruite à partir de notes, de tickets, de l'historique du terminal et de l'expérience du développeur. Avec la Factory, la trajectoire devient un objet du système : elle est découpée en stages, associée à des préconditions et reliée à des validations. Cette formalisation rend l'avancement plus lisible et réduit la dépendance à la mémoire individuelle.

La même différence apparaît dans la gestion des échecs. Lorsqu'une commande échoue manuellement, le développeur doit identifier le contexte pertinent, relire les journaux, rechercher la cause probable, modifier l'environnement ou le code puis relancer les vérifications. La Factory ne supprime pas ce raisonnement, mais elle cherche à préparer automatiquement le contexte nécessaire : commande exécutée, code de sortie, journaux, stage, profil de runtime, fichiers concernés et état de la migration. Les agents peuvent

ensuite exploiter ce contexte pour proposer une interprétation ou un correctif, tandis que les services déterministes conservent l'autorité sur l'application et la revalidation.

Illustration par un cas Angular réel. Un cas observé pendant la transition Angular 19 vers 20 illustre cette différence. La première tentative d'actualisation a échoué sur un conflit de dépendances npm : une dépendance de test bloquait la résolution du paquet de construction Angular cible. Après retrait contrôlé de cette dépendance, une nouvelle tentative a révélé un second problème, lié à des versions de correctif Angular incohérentes dans l'arbre de dépendances.

Dans une approche manuelle, le développeur aurait dû relire les erreurs npm, identifier la dépendance bloquante, modifier le manifeste, relancer l'actualisation, constater le second conflit, normaliser les versions, reconstruire l'installation puis exécuter à nouveau la construction et les tests. Cette démarche est possible, mais le lien entre chaque tentative, la cause de l'échec et le résultat obtenu reste souvent dispersé entre le terminal, les fichiers modifiés et les notes de suivi.

Dans la Factory, chaque échec est conservé avec son contexte d'exécution. La correction a été appliquée dans l'espace de stage isolé, puis l'installation a été reconstruite avec `npm ci`. Le système a ensuite vérifié les versions résolues, exécuté la construction de production et les tests, puis soumis le résultat aux décisions de validation avant de sceller le stage. Le bénéfice ne réside donc pas dans une suppression du raisonnement du développeur, mais dans l'encadrement du diagnostic, la conservation des preuves et l'obligation de revalider le résultat avant de poursuivre.

Ce cas met également en évidence la différence entre une commande réussie et un stage validé. Dans la preuve d'exécution retenue, Angular 18 vers 19 et 19 vers 20 sont scellés ; Angular 20 vers 21 est seulement préparé et reste en attente de l'autorisation de démarrage. La Factory ne présente donc pas une transition comme acquise tant que les validations et le scellement associés ne sont pas enregistrés.

Cette distinction a un impact direct sur la **reproductibilité**. Le résultat d'une migration ne dépend plus uniquement du fait qu'une série de commandes ait terminé avec un code de sortie favorable ; il dépend également du profil utilisé, de l'état de l'espace de travail et des validations associées. En conservant ces informations, la Factory rend possible une analyse a posteriori plus fiable et facilite la comparaison entre deux exécutions.

L'impact sur la **traçabilité** est également important. Dans le processus manuel, le lien entre un échec, la décision prise pour le corriger et le résultat du nouvel essai peut être implicite. Dans la Factory, l'objectif est que ces éléments soient rattachés au même stage et à la même exécution. Cette organisation ne garantit pas qu'un diagnostic agentique soit toujours correct, mais elle réduit le risque qu'une proposition soit appliquée hors contexte ou qu'une décision soit évaluée sur une version différente de l'artefact concerné.

Enfin, la comparaison met en évidence un déplacement du rôle du développeur. Dans le processus manuel, une part importante du temps est consacrée à l'enchaînement des commandes, à la collecte d'informations et à la reconstruction de l'état. Dans

la Factory, ces opérations sont davantage structurées par le système ; l'intervention humaine se concentre sur la validation des choix sensibles, l'examen des propositions et les situations qui dépassent les règles disponibles. Ce déplacement correspond à l'objectif du projet : assister le développeur dans la migration sans lui retirer l'autorité sur les décisions importantes.

Cette analyse reste qualitative. Elle ne permet pas encore d'affirmer qu'une migration réalisée avec la Factory est, par exemple, deux fois plus rapide qu'une migration manuelle. Une telle conclusion exigerait un protocole contrôlé, plusieurs applications comparables, des répétitions et une mesure précise du temps actif du développeur. En revanche, le cas Angular montre déjà de manière concrète que la Factory transforme une succession d'actions techniques en un processus persistant, explicite et réutilisable, ce qui constitue l'apport principal recherché avant même la mesure d'un gain temporel.

Cette comparaison exprime les capacités du système ; elle ne remplace pas une expérience chronométrée sur deux groupes comparables. Une telle étude reste à réaliser avant de quantifier un gain de productivité.

5.8 Limites de l'évaluation

Les limites principales sont les suivantes :

- nombre limité de scénarios complets et reproductibles ;
- absence de benchmark manuel contrôlé ;
- métriques de durée et de coût LLM incomplètes ;
- maturité différente entre Java et Angular ;
- scénarios Maven réels principalement manuels ou en recette utilisateur ;
- absence de chaîne CI commune dans les dépôts des prototypes ;
- absence de politique globale unique de lint ;
- absence d'authentification et de test multi-utilisateur ;
- captures finales et comptes rendus de démonstration encore à intégrer au document.

Ces limites empêchent d'annoncer un pourcentage global de gain de temps ou de réussite. Le rapport conserve donc une distinction entre résultats vérifiés, validation par présentation et bénéfices attendus.

Conclusion

L'évaluation montre que la déclinaison Java valide les principaux mécanismes de gouvernance, d'isolation, de réparation et de traçabilité. Les trois démonstrations Java réalisées matérialisent le passage d'un prototype CLI à une plateforme full stack. Côté Angular, la preuve d'exécution versionnée retenue valide les transitions 18 vers 19 et 19 vers 20, tandis que 20 vers 21 reste préparé mais non démarré. Le parcours complet 11 vers 21 demeure donc un objectif d'extension et ne doit pas être présenté comme un résultat déjà démontré.

Le principal apport du projet est une organisation contrôlée des agents et des outils, plutôt qu'une autonomie totale. Les preuves disponibles sont encourageantes, mais elles ne permettent pas encore de quantifier un gain global. Une campagne expérimentale plus large, reproductible et instrumentée constitue la prochaine étape nécessaire à l'industrialisation.

Conclusion générale

Ce projet de fin d'études avait pour objectif de concevoir et de développer une Migration Factory capable d'assister des migrations logicielles complexes sans déléguer entièrement l'autorité à des modèles probabilistes. La solution retenue combine une orchestration persistante, des agents spécialisés, des services déterministes, des espaces de travail isolés, des preuves techniques et une supervision humaine.

L'étude du processus manuel a montré que la difficulté d'une migration ne réside pas uniquement dans la modification du code. Elle concerne également la préparation des environnements, la compatibilité des versions, l'enchaînement des validations, le diagnostic des échecs et la conservation des décisions. Ces constats ont conduit à une architecture multi-agents hybride et centralement orchestrée.

Le travail réalisé distingue clairement les agents intelligents des services déterministes. Les agents analysent, planifient, révisent, transforment ou proposent des réparations. Les services déterministes conservent l'autorité sur les états, les transitions, les profils, les commandes, les chemins, les écritures, les validations et les rapports. Le développeur reste responsable des décisions sensibles.

L'évolution du projet constitue un résultat important. Une première démonstration Java/Spring Boot, réalisée le 25 mai 2026, a validé la faisabilité d'un workflow en ligne de commande. Une deuxième démonstration, le 3 juin, a présenté une version CLI corrigée et consolidée à une audience CGI plus large. La troisième démonstration, le 19 juin, a marqué le passage à une Control Tower full stack intégrant backend, frontend, persistance, cockpit et chatbot.

La déclinaison Java/Spring Boot constitue ainsi le système le plus mature. Elle valide les principaux mécanismes du projet : trajectoires progressives, points de validation, isolation, compilation et tests, réparation gouvernée, cockpit, assistant et reporting. Les travaux engagés depuis le 20 juin préparent une V1 comprenant un nouveau frontend, une boucle de réparation renforcée et une meilleure exploitation des LLM. Cette démonstration reste planifiée pour septembre 2026 au 30 août 2026.

Le choix d'Azure OpenAI renforce l'intégration de la solution dans le contexte de CGI. La plateforme ne dépend pas d'un modèle unique codé en dur : l'endpoint, l'authentification et les noms de déploiement sont externalisés. CGI peut ainsi connecter les modèles qu'elle a déployés et gouvernés dans sa propre ressource Azure, tout en attribuant des déploiements distincts aux rôles de proposition, de révision, d'assistance et de secours. L'implémentation Java actuelle utilise une authentification par clé API ; l'adoption de Microsoft Entra ID, d'une identité managée et d'un coffre de secrets constitue une évolution d'industrialisation.

La déclinaison Angular montre que les principes peuvent être adaptés à un second écosystème, avec des profils Node.js, npm et TypeScript ainsi que des transformations propres au frontend. Son développement a commencé à la mi-juillet 2026 et l'objectif a

ensuite été étendu à une trajectoire progressive d'Angular 11 vers Angular 21. La preuve d'exécution versionnée retenue valide les transitions Angular 18 vers 19 et 19 vers 20 ; le stage 20 vers 21 est préparé mais non démarré. Le parcours 11 vers 21 demeure donc un objectif d'achèvement et non un résultat déjà démontré. L'utilisation effective d'Azure dans Angular devra également être confirmée par l'audit du dépôt correspondant.

Les résultats disponibles confirment l'intérêt de la séparation entre raisonnement et exécution. Les états et preuves rendent le parcours plus observable, tandis que les empreintes et l'approbation humaine limitent l'application de résultats obsolètes ou non examinés. Les démonstrations successives ont également permis de révéler les limites de chaque incrément et d'orienter les évolutions suivantes.

Néanmoins, l'évaluation quantitative demeure partielle. Les gains de temps, les taux de réussite, la consommation des modèles et le comportement sur un ensemble représentatif d'applications doivent encore être mesurés de manière reproductible. Une démonstration constitue une validation par les parties prenantes ; elle ne remplace pas une campagne expérimentale répétée et instrumentée.

Les perspectives sont organisées selon quatre axes complémentaires.

À court terme, les travaux portent sur l'achèvement du parcours Angular, la consolidation de la réparation gouvernée, du reporting et de l'assistant, ainsi que sur la stabilisation des scénarios de validation de bout en bout.

Pour l'industrialisation, les priorités concernent l'authentification et les rôles, l'utilisation de Microsoft Entra ID et d'Azure Key Vault, la mise en place d'une chaîne d'intégration continue, l'amélioration de la portabilité et l'adaptation de la persistance à des déploiements multi-instance.

Pour l'évaluation, il reste à automatiser la collecte des durées, des appels LLM, des coûts et des interventions humaines, puis à constituer un corpus reproductible de projets Java et Angular afin d'exécuter plusieurs campagnes comparables.

À plus long terme, les réparations validées pourraient être capitalisées sous forme de règles déterministes réutilisables, tandis que de nouveaux profils et adaptateurs permettraient d'étendre la démarche à d'autres versions, frameworks et écosystèmes.

La contribution principale du projet réside ainsi dans une approche gouvernée de l'automatisation : les agents accélèrent l'analyse et la proposition, mais les mécanismes déterministes et le développeur conservent le contrôle de l'exécution. La progression du prototype CLI vers une plateforme full stack, puis vers une V1 gouvernée et une déclinaison Angular, constitue une base crédible pour poursuivre l'industrialisation d'une plateforme générale de migration logicielle.

Bibliographie

- [1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148 :2018 – Systems and software engineering – Life cycle processes – Requirements engineering*, 2^e édition, 2018, confirmée en 2024. <https://www.iso.org/standard/72089.html>
- [2] ISO/IEC/IEEE, *ISO/IEC/IEEE 42010 :2022 – Software, systems and enterprise – Architecture description*, 2^e édition, 2022. <https://www.iso.org/standard/74393.html>
- [3] ISO/IEC, *ISO/IEC 25010 :2023 – Systems and software Quality Requirements and Evaluation – Product quality model*, 2023. <https://www.iso.org/standard/78176.html>
- [4] ISO/IEC, *ISO/IEC 23894 :2023 – Information technology – Artificial intelligence – Guidance on risk management*, 2023. <https://www.iso.org/standard/77304.html>
- [5] ISO/IEC, *ISO/IEC 27001 :2022 – Information security, cybersecurity and privacy protection – Information security management systems*, 2022. <https://www.iso.org/standard/27001.html>
- [6] ISO, *ISO 21502 :2020 – Project, programme and portfolio management – Guidance on project management*, 2020. <https://www.iso.org/standard/74947.html>
- [7] Object Management Group, *OMG Unified Modeling Language, Version 2.5.1*, 2017. <https://www.omg.org/spec/UML/2.5.1>
- [8] M. Wooldridge et N. R. Jennings, *Intelligent Agents : Theory and Practice*, The Knowledge Engineering Review, vol. 10, no. 2, pp. 115–152, 1995. <https://www.cambridge.org/core/journals/knowledge-engineering-review/article/intelligent-agents-theory-and-practice/CF2A6AAEEA1DBD486EF019F6217F1597>
- [9] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2^e édition, John Wiley & Sons, 2009. <https://www.cs.ox.ac.uk/people/michael.wooldridge/pubs/imas/>
- [10] G. Weiss (dir.), *Multiagent Systems*, 2^e édition, MIT Press, 2016. <https://mitpress.mit.edu/9780262533874/multiagent-systems/>
- [11] J. Ferber, *Multi-Agent Systems : An Introduction to Distributed Artificial Intelligence*, Addison-Wesley, 1999.
- [12] Foundation for Intelligent Physical Agents, *FIPA Agent Communication Language Specifications*. <https://www.fipa.org/repository/aclspecs.html>
- [13] World Wide Web Consortium, *Web Services Choreography Description Language Version 1.0*. <https://www.w3.org/TR/ws-cdl-10/>
- [14] K. Schwaber et J. Sutherland, *The Scrum Guide : The Definitive Guide to Scrum*, novembre 2020. <https://scrumguides.org/scrum-guide.html>

- [15] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1, 2023. <https://www.nist.gov/itl/ai-risk-management-framework>
- [16] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework : Generative Artificial Intelligence Profile*, NIST AI 600-1, 2024. <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- [17] Microsoft, *Endpoints for Microsoft Foundry Models*, documentation officielle, consultée en août 2026. <https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/endpoints>
- [18] Microsoft, *Azure OpenAI supported programming languages and authentication methods*, documentation officielle, consultée en août 2026. <https://learn.microsoft.com/en-us/azure/foundry/openai/supported-languages>
- [19] Microsoft, *Configure keyless authentication with Microsoft Entra ID*, documentation officielle, consultée en août 2026. <https://learn.microsoft.com/en-us/azure/foundry/foundry-models/how-to/configure-entra-id>
- [20] Microsoft, *Apps, API Keys, and Azure Key Vault secrets*, documentation officielle, consultée en août 2026. <https://learn.microsoft.com/en-us/azure/key-vault/general/apps-api-keys-secrets>
- [21] CGI Inc., *CGI at a glance*, consulté en août 2026. <https://www.cgi.com/en/about-us/cgi-at-a-glance>
- [22] CGI Inc., *CGI history*, consulté en août 2026. <https://www.cgi.com/en/about-us/cgi-history>
- [23] CGI Maroc, *Présentation des activités et implantations au Maroc*, consulté en août 2026. <https://www.cgi.com/maroc/fr-fr>
- [24] FastAPI, *Dependencies – Dependency Injection*, documentation officielle, consultée en août 2026. <https://fastapi.tiangolo.com/tutorial/dependencies/>
- [25] LangChain, *LangGraph Persistence*, documentation officielle, consultée en août 2026. <https://docs.langchain.com/oss/python/langgraph/persistence>
- [26] LangChain, *Multi-agent systems and orchestration patterns*, documentation officielle, consultée en août 2026. <https://docs.langchain.com/oss/python/langchain/multi-agent/index>
- [27] SQLite, *Write-Ahead Logging*, documentation officielle. <https://sqlite.org/wal.html>
- [28] OpenRewrite, *Recipes*, documentation officielle. <https://docs.openrewrite.org/concepts-and-explanations/recipes>
- [29] OpenRewrite, *Recipe conventions and best practices*, documentation officielle. <https://docs.openrewrite.org/authoring-recipes/recipe-conventions-and-best-practices>
- [30] Spring, *Upgrading Spring Boot Applications*, documentation officielle. <https://docs.spring.io/spring-boot/upgrading.html>

- [31] Angular, *Version compatibility*, documentation officielle. <https://angular.dev/reference/versions>
- [32] Angular, *Keeping your projects up-to-date and Angular Update Guide*, documentation officielle. <https://angular.dev/update>
- [33] Angular, *Versioning and releases – Breaking change policy and update paths*, documentation officielle, consultée en août 2026. <https://angular.dev/reference/releases>
- [34] Angular, *Building Angular applications*, documentation officielle. <https://angular.dev/tools/cli/build>
- [35] Angular, *ng test*, documentation officielle. <https://angular.dev/cli/test>
- [36] GitHub, *Dependabot version updates*, documentation officielle. <https://docs.github.com/en/code-security/concepts/supply-chain-security/dependabot-version-updates>
- [37] Amazon Web Services, *Upgrading Java versions with Amazon Q Developer*, documentation officielle, consultée en août 2026. <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/code-transformation.html>
- [38] Microsoft, *GitHub Copilot Modernization Documentation*, documentation officielle, consultée en août 2026. <https://learn.microsoft.com/en-us/azure/developer/github-copilot-app-modernization/>
- [39] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan et O. Press, *SWE-agent : Agent-Computer Interfaces Enable Automated Software Engineering*, Advances in Neural Information Processing Systems, 2024. <https://arxiv.org/abs/2405.15793>
- [40] Q. Wu et al., *AutoGen : Enabling Next-Gen LLM Applications via Multi-Agent Conversation*, Conference on Language Modeling (COLM), 2024 ; Best Paper, LLM Agents Workshop, ICLR 2024. <https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/>
- [41] M. Monperrus, S. Urli, T. Durieux, M. Martinez, B. Baudry et L. Seinturier, *Human-competitive Patches in Automatic Program Repair with Repairnator*, 2018. <https://arxiv.org/abs/1810.05806>